

LAZY EVALUATION

ISIS 4217

HOW ARE EXPRESSIONS EVALUATED



Eager evaluation

```
fun {Append Xs Y}
   case Xs of nil then Y
   [] X|Xr then X|{Append Xr Y}
   end
end
{Append 1|2|3|nil 4|5|nil}
```



1. Evaluate the arguments
2. Evaluate the procedure
3. Apply the arguments to the procedure

Eager evaluation

```
{Append 1|2|3|nil 4|5|nil}
```

1. Evaluate the cons’

```
fun {Append Xs Y}
   case Xs of nil then Y
   [] X|Xr then X|{Append Xr Y}
   end
end
{Append [1 2 3] [4 5]}
```

Eager evaluation

```
{Append 1|2|3|nil 4|5|nil}
```

1. Evaluate the cons'

```
   1|2|3|nil
=  1|2|[3]
=  1|[2 3]
=  [1 2 3]
```

```
fun {Append Xs Y}
   case Xs of nil then Y
   [] X|Xr then X|{Append Xr Y}
   end
end
{Append [1 2 3] [4 5]}
```

Eager evaluation

```
{Append 1|2|3|nil 4|5|nil}
```

1. Evaluate the cons'

```
   1|2|3|nil
=  1|2|[3]
=  1|[2 3]
=  [1 2 3]
```

```
   4|5|nil
=  4|[5]
=  [4 5]
```

```
fun {Append Xs Y}
   case Xs of nil then Y
   [] X|Xr then X|{Append Xr Y}
   end
end
{Append [1 2 3] [4 5]}
```

Eager evaluation

```
{Append 1|2|3|nil 4|5|nil}
```

1. Evaluate the cons'

```
   1|2|3|nil
=  1|2|[3]
=  1|[2 3]
=  [1 2 3]
```

one evaluation per cons

```
   4|5|nil
=  4|[5]
=  [4 5]
```

```
fun {Append Xs Y}
   case Xs of nil then Y
   [] X|Xr then X|{Append Xr Y}
   end
end
{Append [1 2 3] [4 5]}
```

Eager evaluation

```
{Append [1 2 3] [4 5]}
```

2. Evaluate the procedure

```
<Append> ↦
  case Xs of nil then Y
  [] X|Xr then X|{Append Xr Y}
  end
```

Eager evaluation

```
{Append [1 2 3] [4 5]}
```

3. Apply the arguments to the procedure

execution order

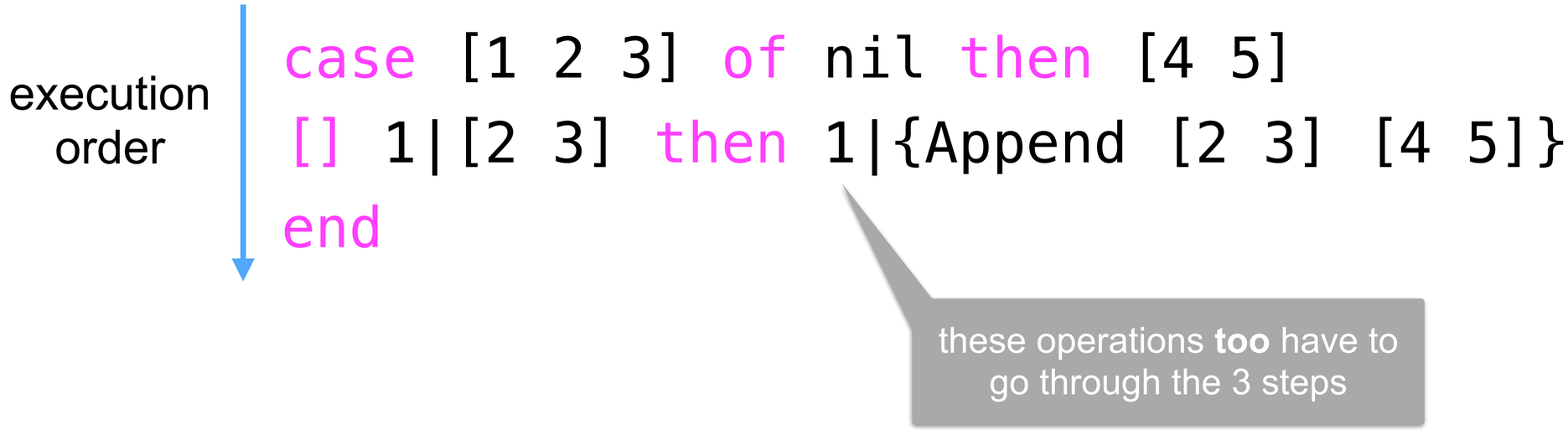


```
case [1 2 3] of nil then [4 5]
[] 1|[2 3] then 1|{Append [2 3] [4 5]}
end
```

Eager evaluation

```
{Append [1 2 3] [4 5]}
```

3. Apply the arguments to the procedure



Eager evaluation

Both cases are evaluated regardless of the input parameters

```
fun {Append Xs Y}
   case Xs of nil then Y
   [] X|Xr then X|{Append Xr Y}
   end
end
{Append 1|2|3|nil 4|5|nil}
```

Eager evaluation

Both cases are evaluated regardless of the input parameters

```
fun {Append Xs Y}
   case Xs of nil then Y
   [] X|Xr then X|{Append Xr Y}
   end
end
{Append 1|2|3|nil 4|5|nil}
```

!!!!

Function evaluation

```
fun {Three X}
   3
end

fun {Infinity}
   {Infinity} + 1
end
```

Function evaluation

```
fun {Three X}
 3
end

fun {Infinity}
 {Infinity} + 1
end
```

```
   {Three 8+5}
= {Three 13}
= 3
```

Function evaluation

```
fun {Three X}
 3
end

fun {Infinity}
 {Infinity} + 1
end
```

```
  {Three 8+5}
= {Three 13}
= 3

  {Infinity}
= {Infinity} + 1
= ({Infinity} + 1) + 1
= (({Infinity} + 1) + 1) + 1
...
```

Function evaluation

```
{Three {Infinity}}
```

Do it yourself

Function evaluation

```
{Three {Infinity}}
```

Do it yourself

Function evaluation

Do it yourself

```
{Three {Infinity}}

  {Three {Infinity}}
```

Function evaluation

Do it yourself

```
{Three {Infinity}}

  {Three {Infinity}}
= {Three ({Infinity}+1)}
```

Function evaluation

Do it yourself

```
{Three {Infinity}}

  {Three {Infinity}}
= {Three ({Infinity}+1)}
= {Three (({Infinity}+1)+1)}
```

Function evaluation

Do it yourself

```
{Three {Infinity}}

  {Three {Infinity}}
= {Three ({Infinity}+1)}
= {Three (({Infinity}+1)+1)}
= {Three ((({Infinity}+1)+1)+1)}
```

Function evaluation

Do it yourself

```
{Three {Infinity}}

  {Three {Infinity}}
= {Three ({Infinity}+1)}
= {Three (({Infinity}+1)+1)}
= {Three ((({Infinity}+1)+1)+1)}
= {Three ((((({Infinity}+1)+1)+1)+1)}
```

Function evaluation

Do it yourself

```
{Three {Infinity}}

  {Three {Infinity}}
= {Three ({Infinity}+1)}
= {Three (({Infinity}+1)+1)}
= {Three ((({Infinity}+1)+1)+1)}
= {Three ((((({Infinity}+1)+1)+1)+1)}
= {Three (((((({Infinity}+1)+1)+1)+1)+1)}
```

Function evaluation

Do it yourself

```
{Three {Infinity}}

  {Three {Infinity}}
= {Three ({Infinity}+1)}
= {Three (({Infinity}+1)+1)}
= {Three ((({Infinity}+1)+1)+1)}
= {Three ((((({Infinity}+1)+1)+1)+1)}
= {Three (((((({Infinity}+1)+1)+1)+1)+1)}
= {Three ((((((({Infinity}+1)+1)+1)+1)+1)+1)}
```

Function evaluation

Do it yourself

```
{Three {Infinity}}

  {Three {Infinity}}
= {Three ({Infinity}+1)}
= {Three (({Infinity}+1)+1)}
= {Three ((({Infinity}+1)+1)+1)}
= {Three ((((({Infinity}+1)+1)+1)+1)}
= {Three (((((({Infinity}+1)+1)+1)+1)+1)}
= {Three ((((((({Infinity}+1)+1)+1)+1)+1)+1)}
= {Three (((((((({Infinity}+1)+1)+1)+1)+1)+1)+1)}
...
```

Function evaluation

Do it yourself

```
{Three {Infinity}}

  {Three {Infinity}}
= {Three ({Infinity}+1)}
= {Three (({Infinity}+1)+1)}
= {Three ((({Infinity}+1)+1)+1)}
= {Three ((((({Infinity}+1)+1)+1)+1)+1)}
= {Three (((((({Infinity}+1)+1)+1)+1)+1)+1)}
= {Three ((((((({Infinity}+1)+1)+1)+1)+1)+1)+1)}
= {Three (((((((({Infinity}+1)+1)+1)+1)+1)+1)+1)+1)}
...
```

How many times do you evaluate Three?

Delayed computations

The idea of lazy evaluation is to **only** evaluate an statement whenever its **result is needed** by another **computation**

Delayed computations

```
B = {LazyFunction1 X}
C = {LazyFunction2 Y}
A = B + C
{Show A}
```

Delayed computations

```
B = {LazyFunction1 X}
C = {LazyFunction2 Y}
A = B + C
{Show A}
```

Do nothing

Delayed computations

```
B = {LazyFunction1 X}
C = {LazyFunction2 Y}
A = B + C
{Show A}
```

Do nothing

Do nothing

Delayed computations

```
B = {LazyFunction1 X}
C = {LazyFunction2 Y}
A = B + C
{Show A}
```

Do nothing

Do nothing

1.evaluate X
2.evaluate LazyFunction1
3.store result in B
4.evaluate Y
5.evaluate LazyFunction2
6.store result in C
7.evaluate +
8.store result in A

Delayed computations

Do it yourself

```
B = {LazyFunction1 X}
C = {LazyFunction2 Y}
A = B
```

Delayed computations

Do it yourself

```
B = {LazyFunction1 X}
C = {LazyFunction2 Y}
A = B
```

Delayed computations

Do it yourself

```
B = {LazyFunction1 X}
C = {LazyFunction2 Y}
A = B
```

Do nothing

Delayed computations

Do it yourself

```
B = {LazyFunction1 X}
C = {LazyFunction2 Y}
A = B
```

Do nothing

Do nothing

Delayed computations

Do it yourself

```
B = {LazyFunction1 X}
C = {LazyFunction2 Y}
A = B
```

Do nothing

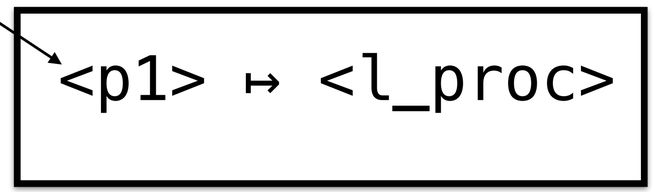
Do nothing

Do nothing

Delayed computations

```
B = {LazyFunction1 X}
C = {LazyFunction2 Y}
A = B + C
```

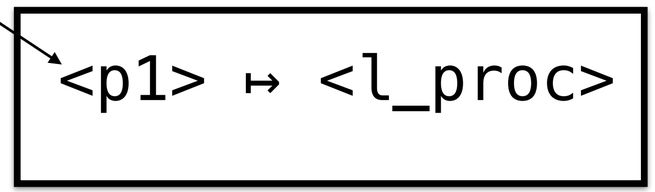
```
fun lazy {LazyFunction1 X}
   1 + X * (3+X*(3+X))
end
```



Delayed computations

```
B = {LazyFunction1 X}
C = {LazyFunction2 Y}
A = B + C
```

```
fun lazy {LazyFunction1 X}
   1 + X * (3+X*(3+X))
end
```



"Nothing" changes in the definition, the difference is in the **`eval`** model

Evaluation

There are **three** different evaluation models for **lazy computation**

1. Lazy functional programming
2. Lazy evaluation with dataflow variables
3. **Demand driven concurrent model**

Evaluation

```
⟨s⟩  ::=  skip
 | thread ⟨s1⟩ end
 | {ByNeed fun{$} ⟨e⟩ end ⟨x⟩ }
```

Lazy functions create a **thunk**

```
fun lazy {LazyFunction1 X}
   1 + X * (3+X*(3+X))
end
```

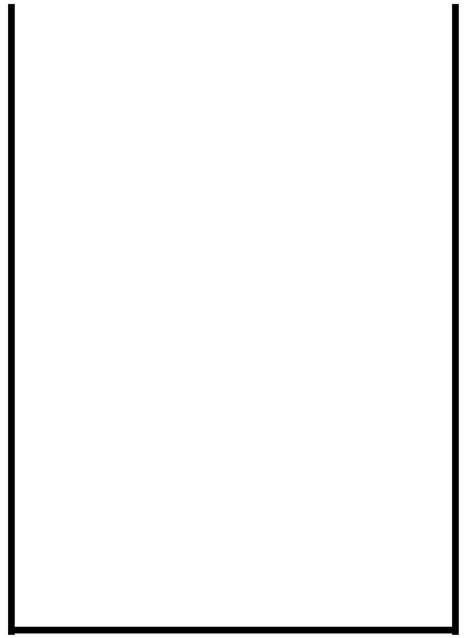


```
fun {LazyFunction1 X}
   {ByNeed fun {$} 1 + X * (3+X*(3+X)) end}
end
```

Evaluation

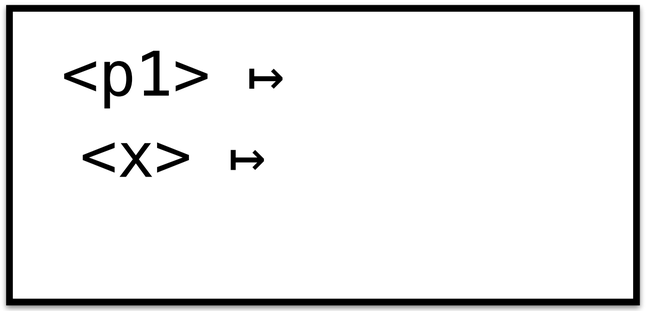
```
fun {LazyFunction1 X}
   {ByNeed fun {$} 1 + X * (3+X*(3+X)) end}
end
B = {LazyFunction1 1}
C = B+1
```

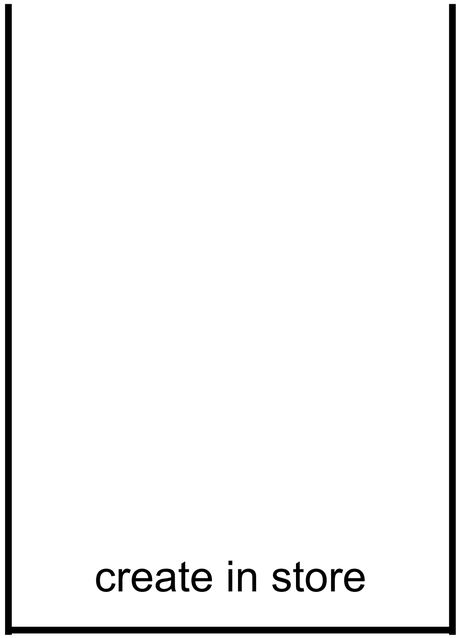




Evaluation

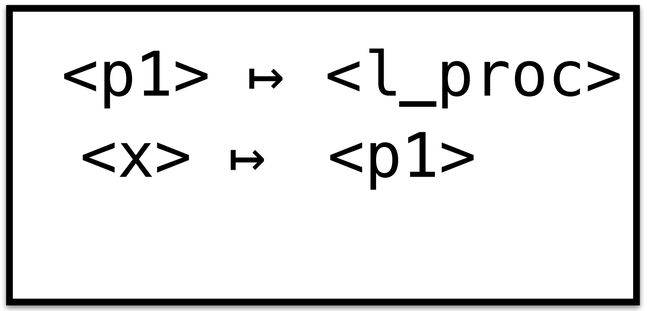
```
fun {LazyFunction1 X}
   {ByNeed fun {$} 1 + X * (3+X*(3+X)) end}
end
B = {LazyFunction1 1}
C = B+1
```





Evaluation

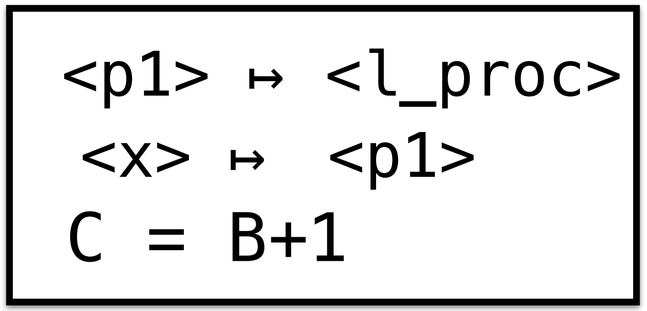
```
fun {LazyFunction1 X}
   {ByNeed fun {$} 1 + X * (3+X*(3+X)) end}
end
B = {LazyFunction1 1}
C = B+1
```

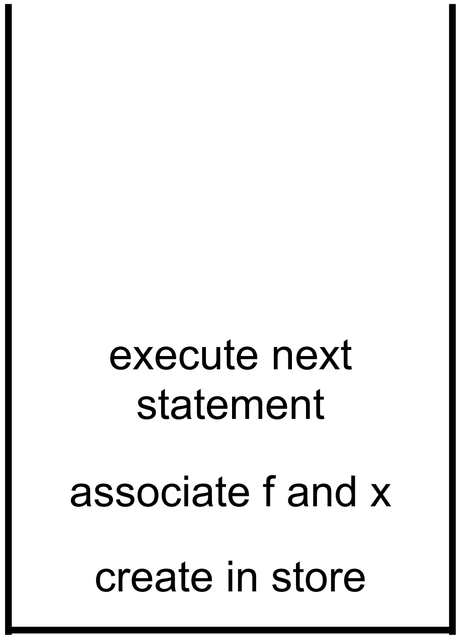




Evaluation

```
fun {LazyFunction1 X}
   {ByNeed fun {$} 1 + X * (3+X*(3+X)) end}
end
B = {LazyFunction1 1}
C = B+1
```





Semantics

```
{ByNeed fun {$} 1 + X * (3+X*(3+X)) end}
```



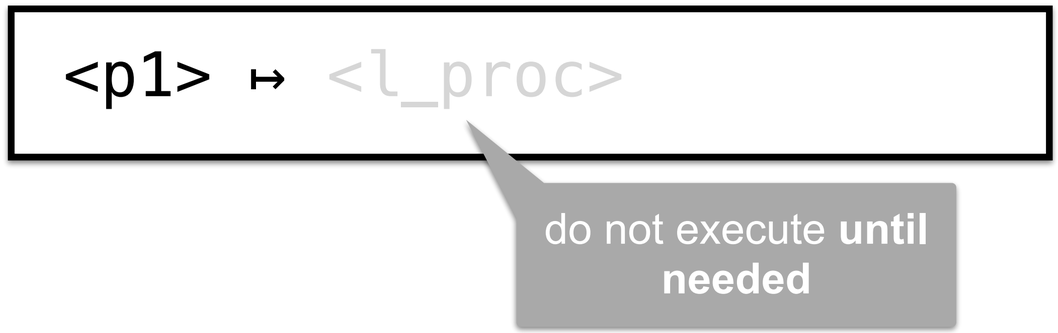
Semantics

```
{ByNeed fun {$} 1 + X * (3+X*(3+X)) end}
```



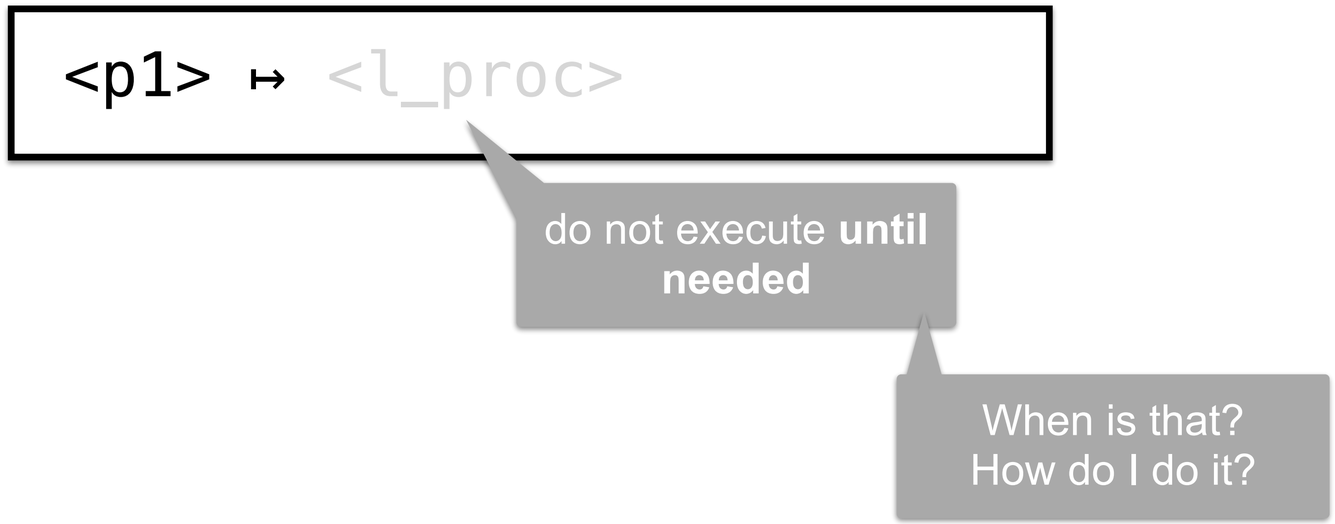
Semantics

```
{ByNeed fun {$} 1 + X * (3+X*(3+X)) end}
```



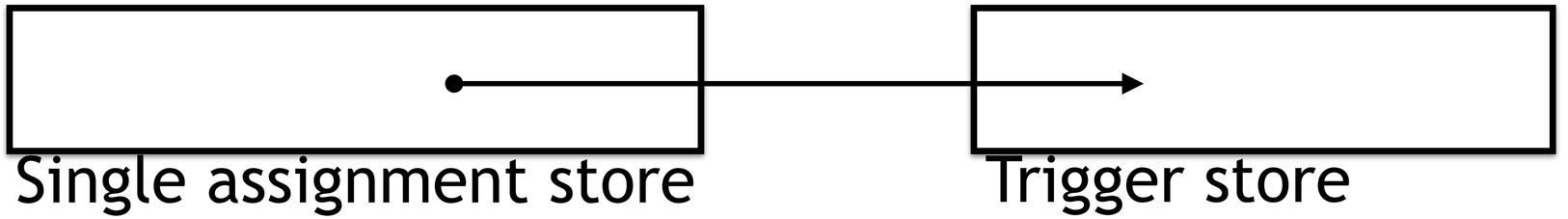
Semantics

```
{ByNeed fun {$} 1 + X * (3+X*(3+X)) end}
```



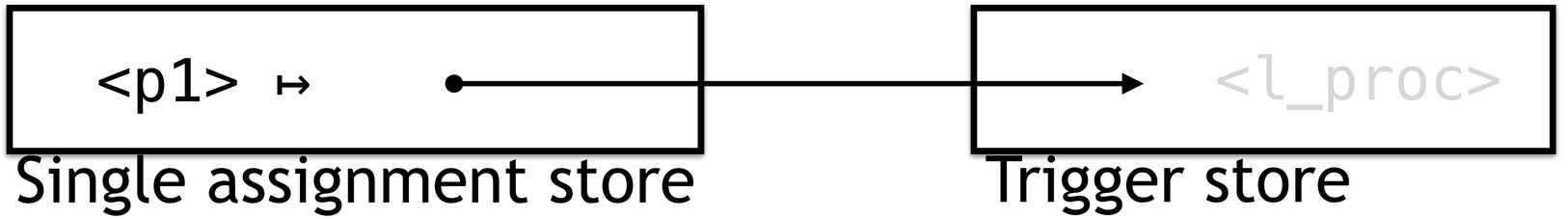
Semantics

```
{ByNeed fun {$} 1 + X * (3+X*(3+X)) end}
```



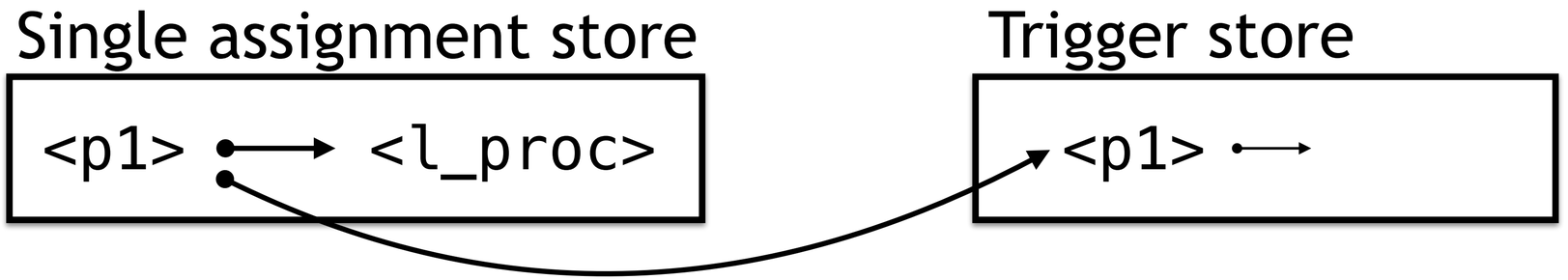
Semantics

```
{ByNeed fun {$} 1 + X * (3+X*(3+X)) end}
```



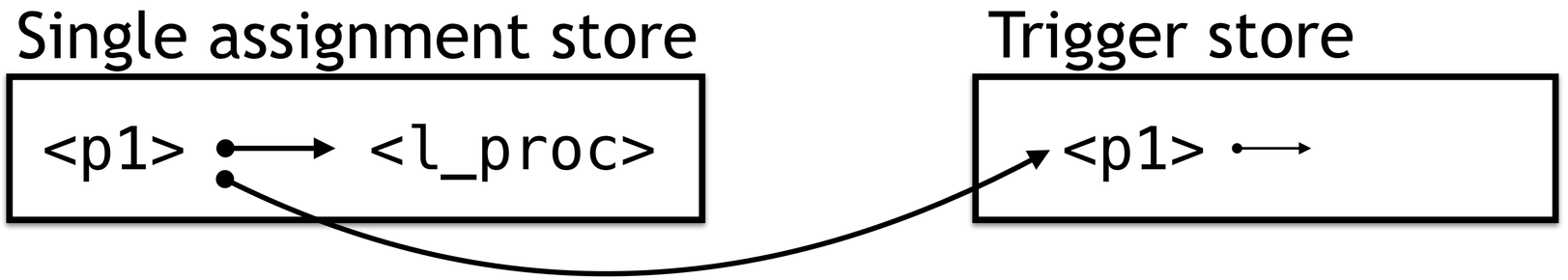
Semantics

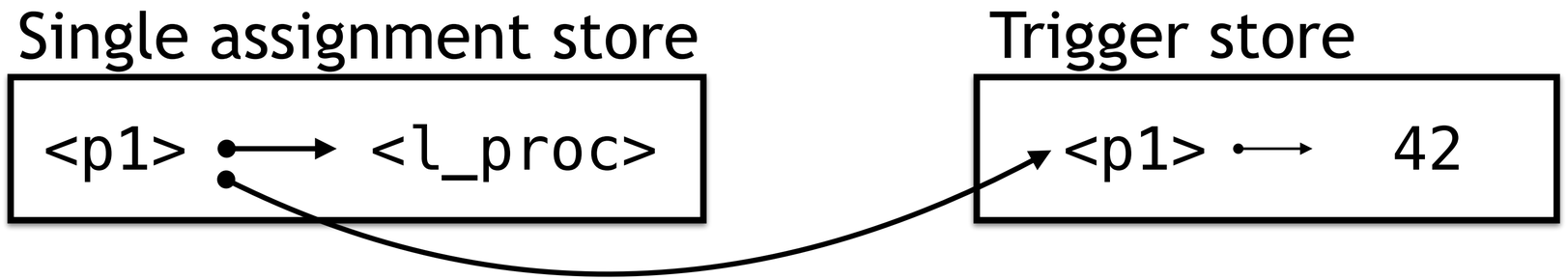
```
{ByNeed fun {$} 1 + X * (3+X*(3+X)) end}
```



Semantics

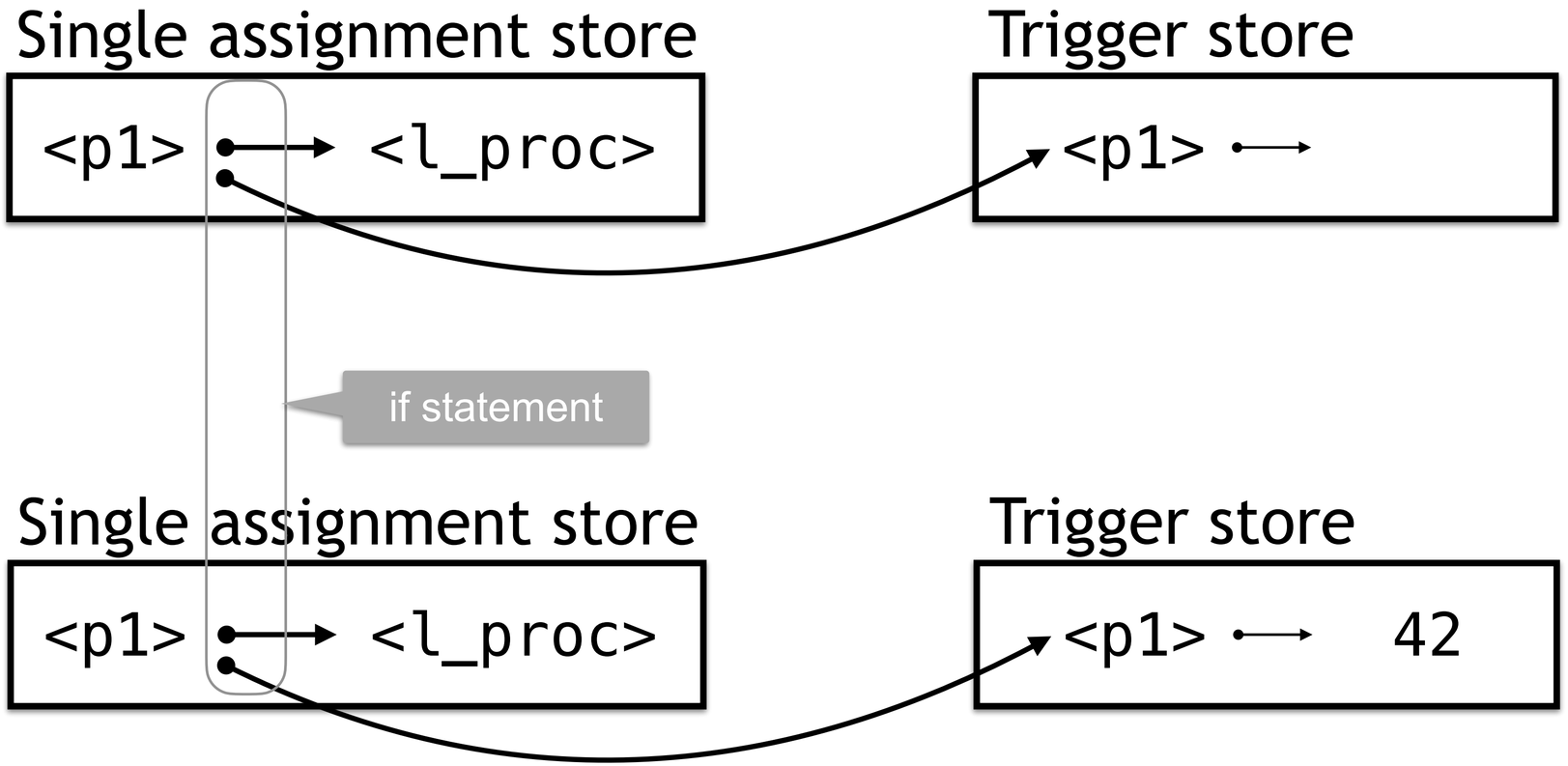
```
{ByNeed fun {$} 1 + X * (3+X*(3+X)) end}
```



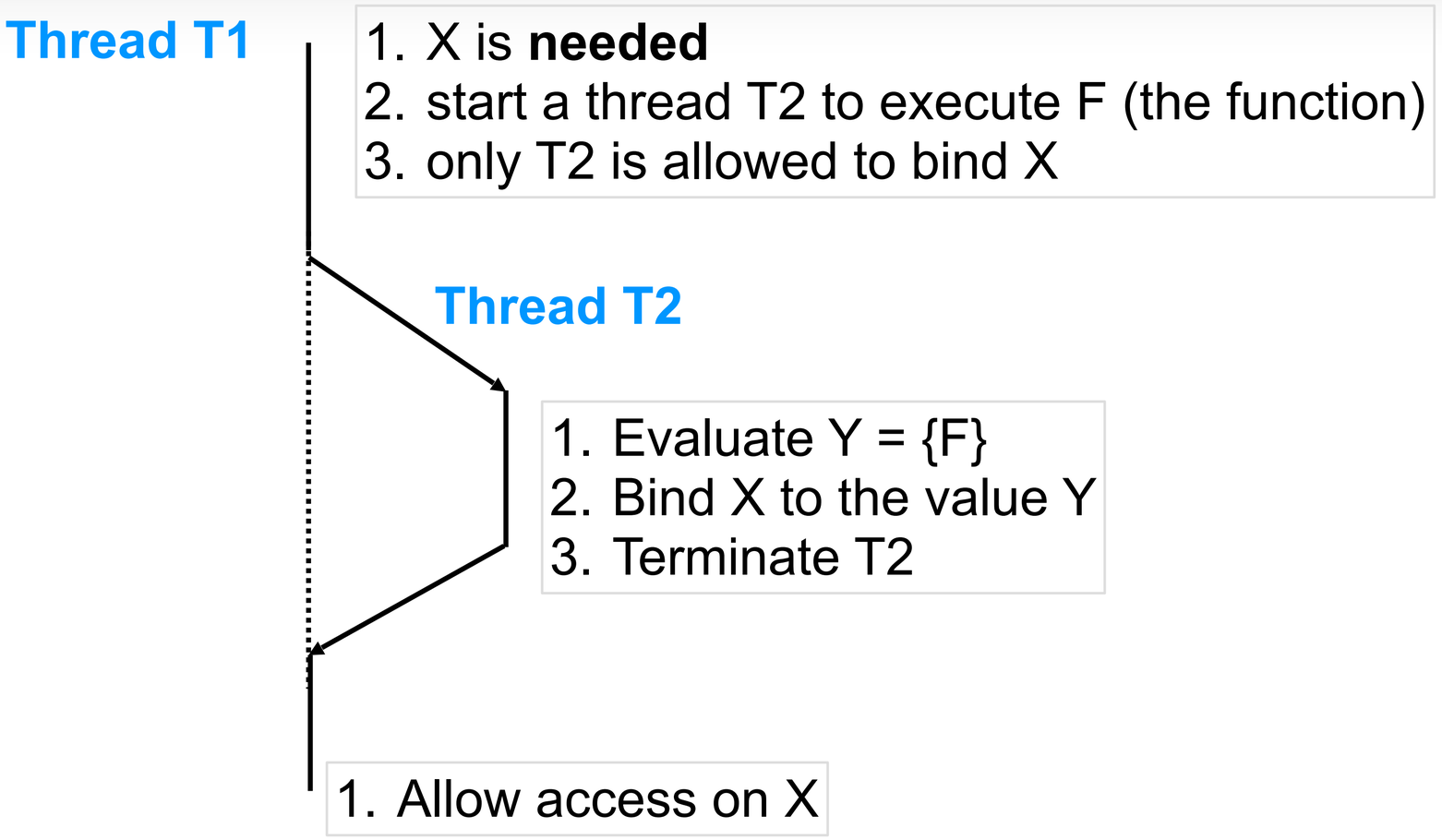


Semantics

```
{ByNeed fun {$} 1 + X * (3+X*(3+X)) end}
```

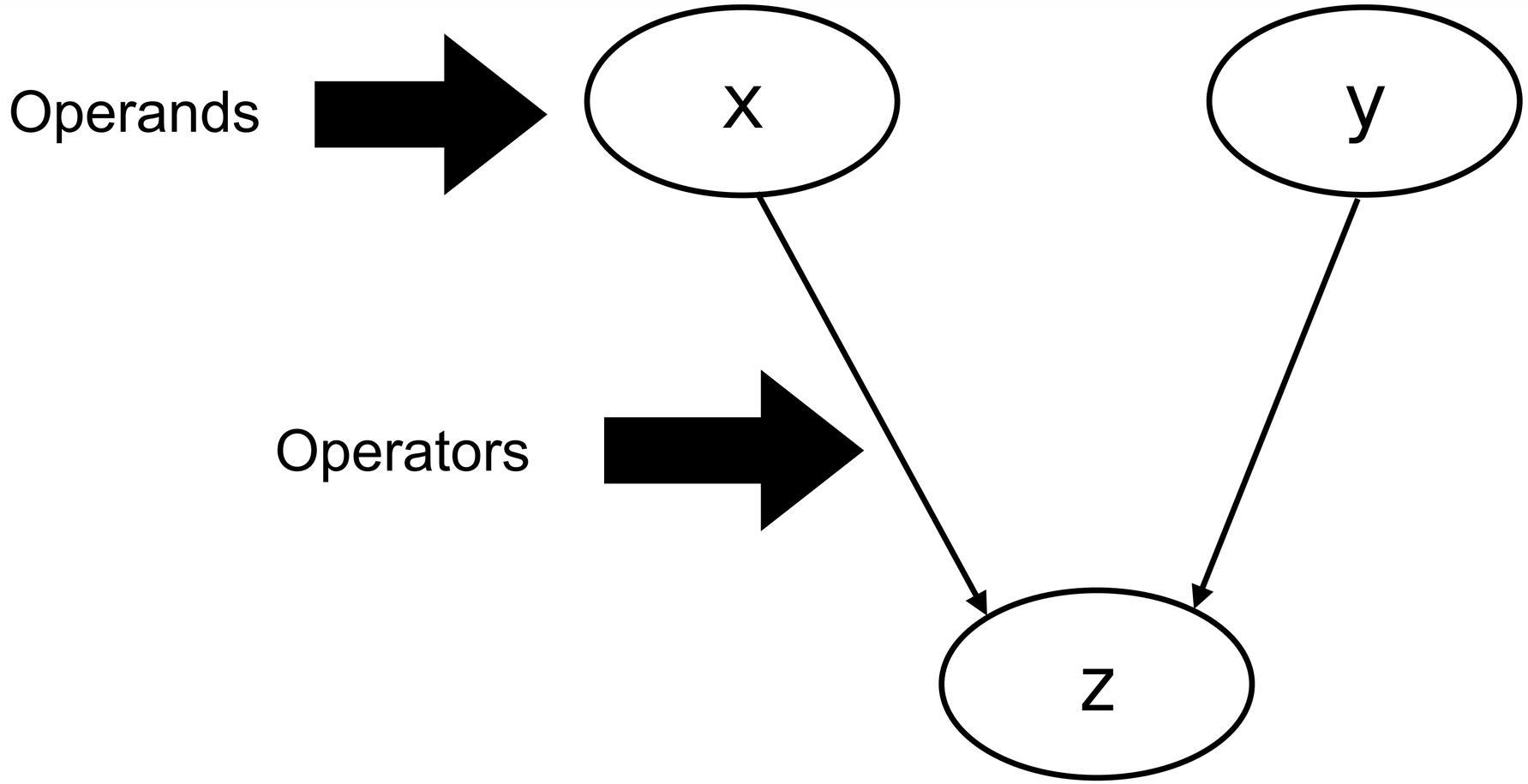


Evaluation

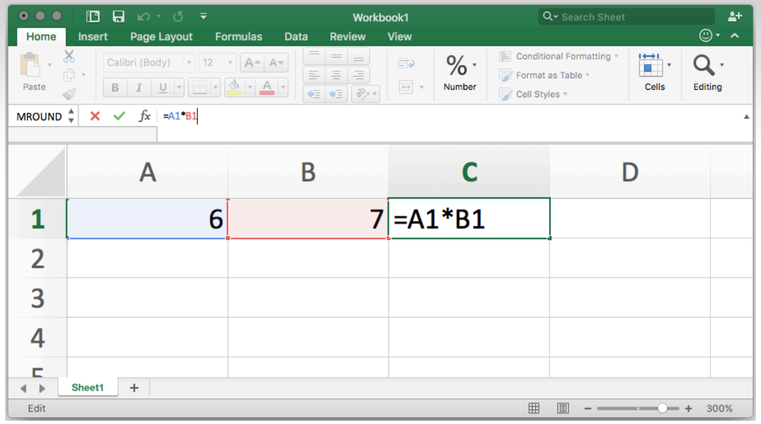

Thread T1
1. X is **needed**
2. start a thread T2 to execute F (the function)
3. only T2 is allowed to bind X
Thread T2
1. Evaluate Y = {F}
2. Bind X to the value Y
3. Terminate T2
1. Allow access on X

Dataflow computation

A direct example of lazy evaluation is dataflow programming

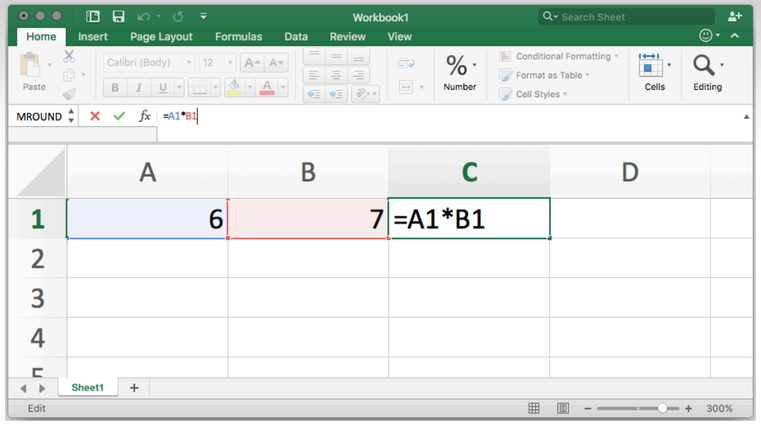


Dataflow computation



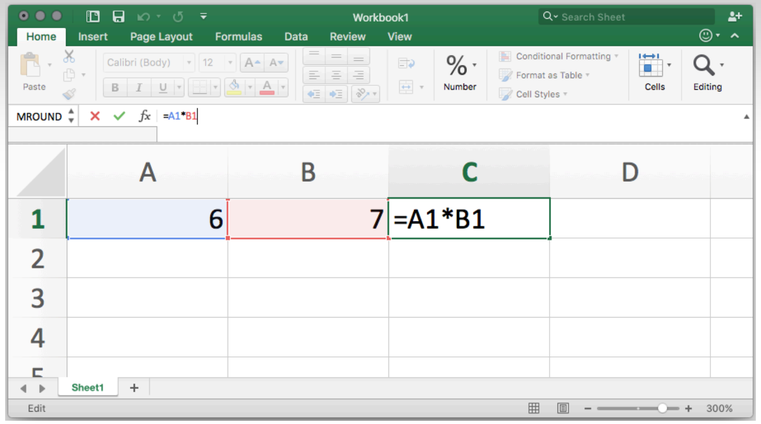

```
A1 = 6
B1 = 7
C1 = A1 * B1
```

Dataflow computation




```
A1 = 6
B1 = 7
C1 = A1 * B1        > C1 = 42
```

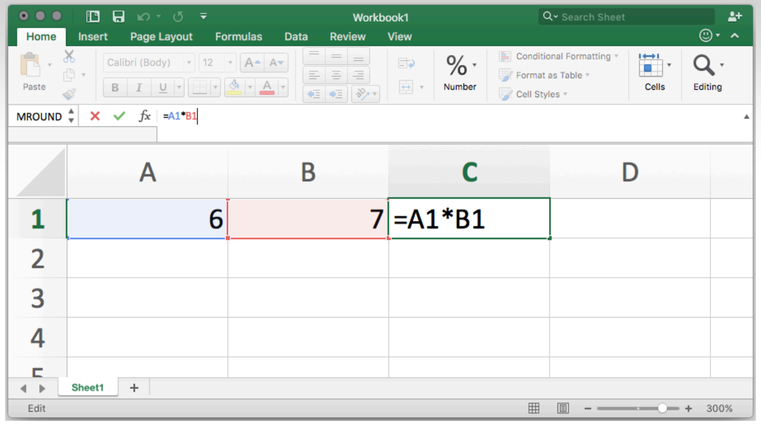
Dataflow computation




```
A1 = 6
B1 = 7
C1 = A1 * B1        > C1 = 42
```

```
A1 = 10
```

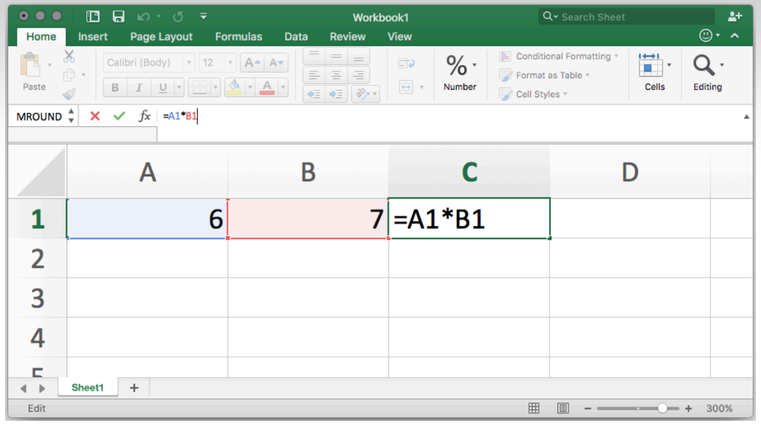
Dataflow computation




```
A1 = 6
B1 = 7
C1 = A1 * B1        > C1 = 42
```

```
A1 = 10
                    > C1 =
```

Dataflow computation

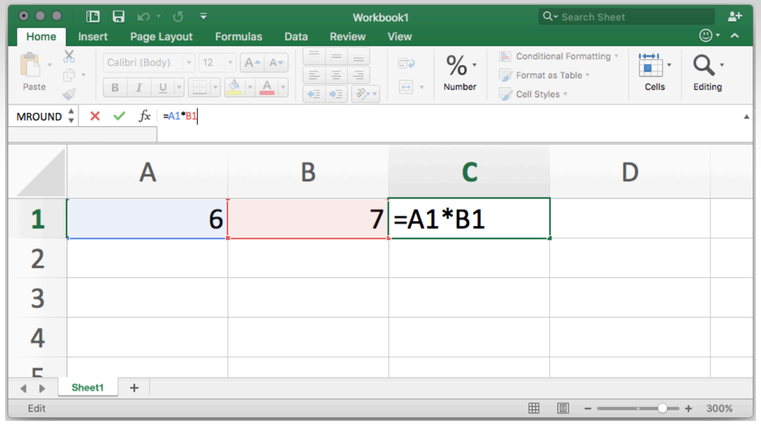



```
A1 = 6
B1 = 7
C1 = A1 * B1        > C1 = 42
```

Is it an assigned variable?

```
A1 = 10
                    > C1 =
```

Dataflow computation

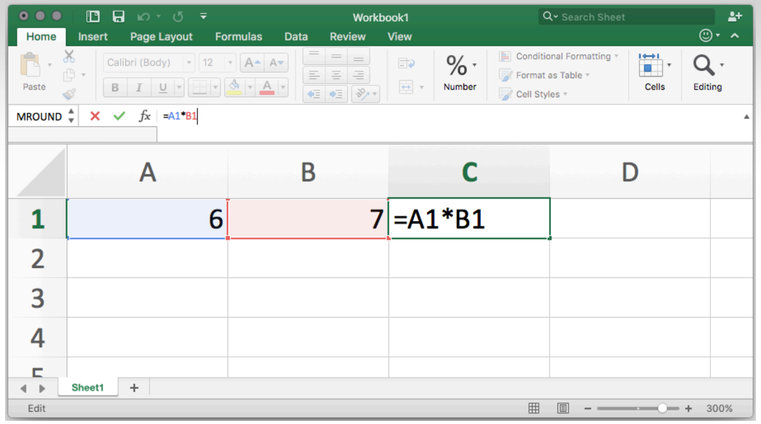



```
A1 = 6
B1 = 7
C1 = A1 * B1        > C1 = 42
```

Is it an assigned variable?

```
A1 = 10        > C1 = 42
```

Dataflow computation




```
A1 = 6
B1 = 7
C1 = A1 * B1
```

```
> C1 = 42
```

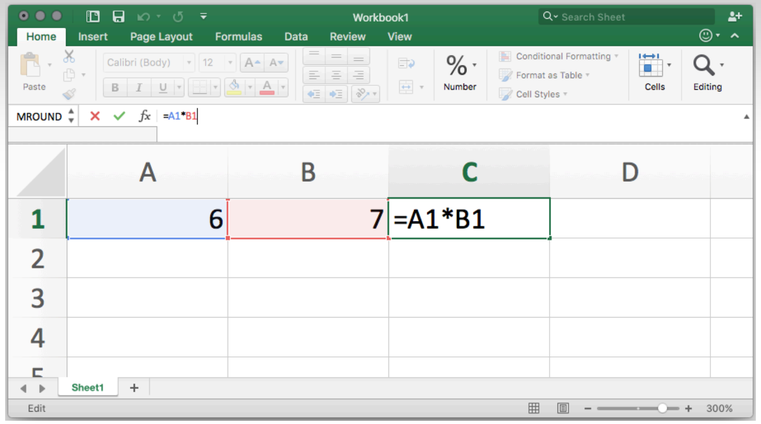
Is it an assigned variable?

Is it the product of A1 and B1?

```
A1 = 10
```

```
> C1 = 42
```

Dataflow computation




```
A1 = 6
B1 = 7
C1 = A1 * B1
```

```
> C1 = 42
```

Is it an assigned variable?

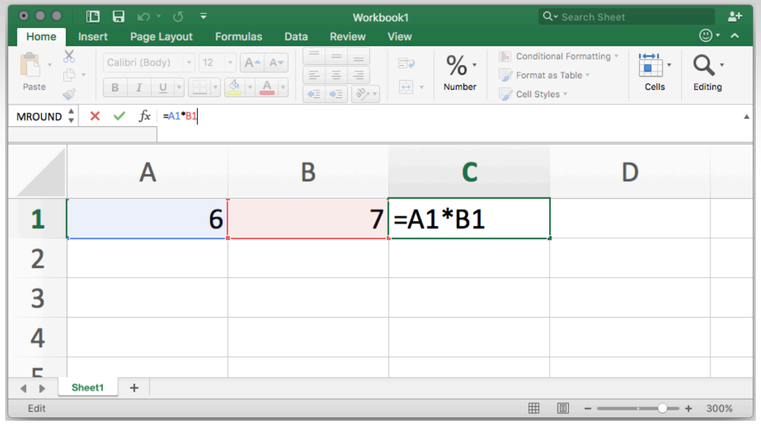
Is it the product of A1 and B1?

```
A1 = 10
```

```
> C1 = 42
```

70

Dataflow computation




```
A1 = 6
B1 = 7
C1 = A1 * B1
```

```
> C1 = 42
```

Is it an assigned variable?

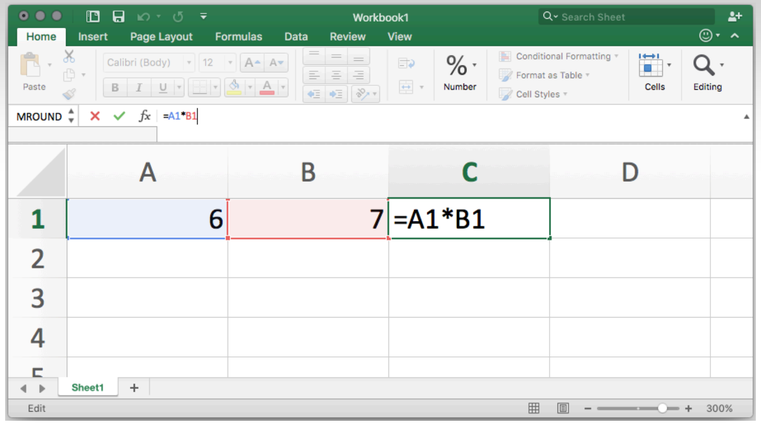
Is it the product of A1 and B1?

```
A1 = 10
```

```
> C1 =
> C1 = 42
```

```
70
```

Dataflow computation




```
A1 = 6
B1 = 7
C1 = A1 * B1
```

```
> C1 = 42
```

Is it an assigned variable?

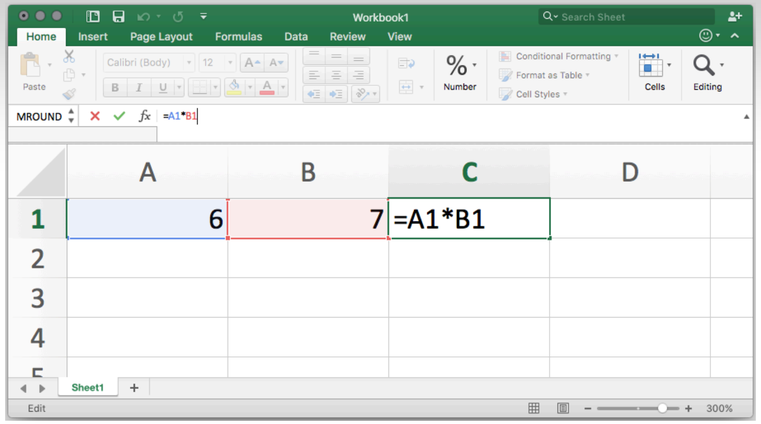
Is it the product of A1 and B1?

```
A1 = 10
```

```
> C1 = 42
> C1 = 42
```

```
70
```

Dataflow computation




```
A1 = 6
B1 = 7
C1 = A1 * B1
```

```
> C1 = 42
```

Is it an assigned variable?

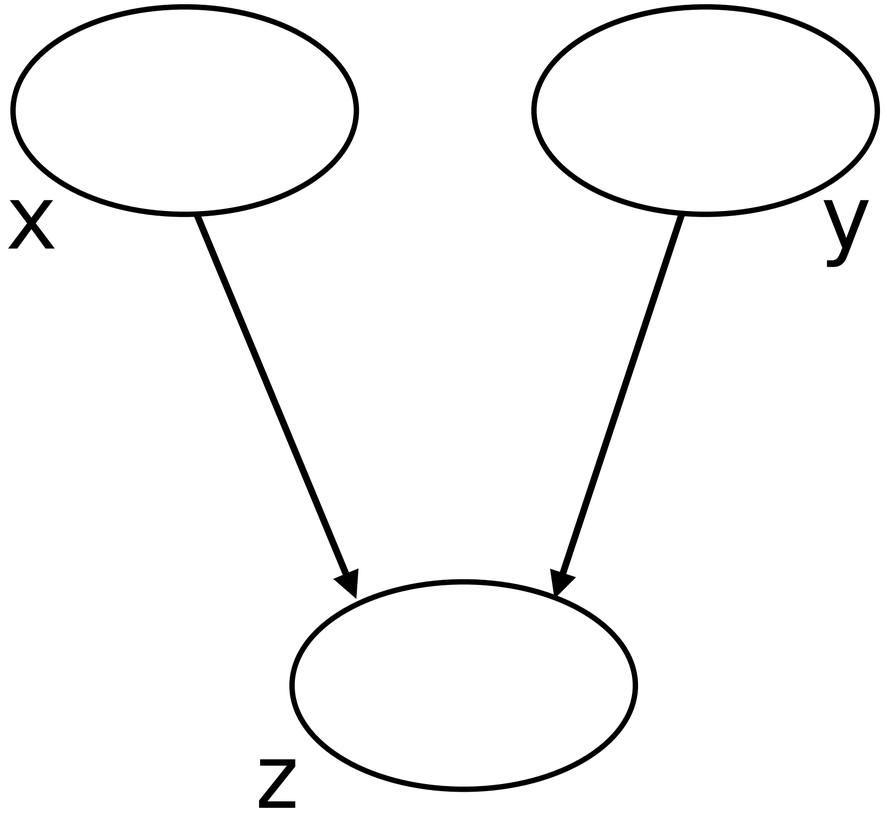
Is it the product of A1 and B1?

```
A1 = 10
```

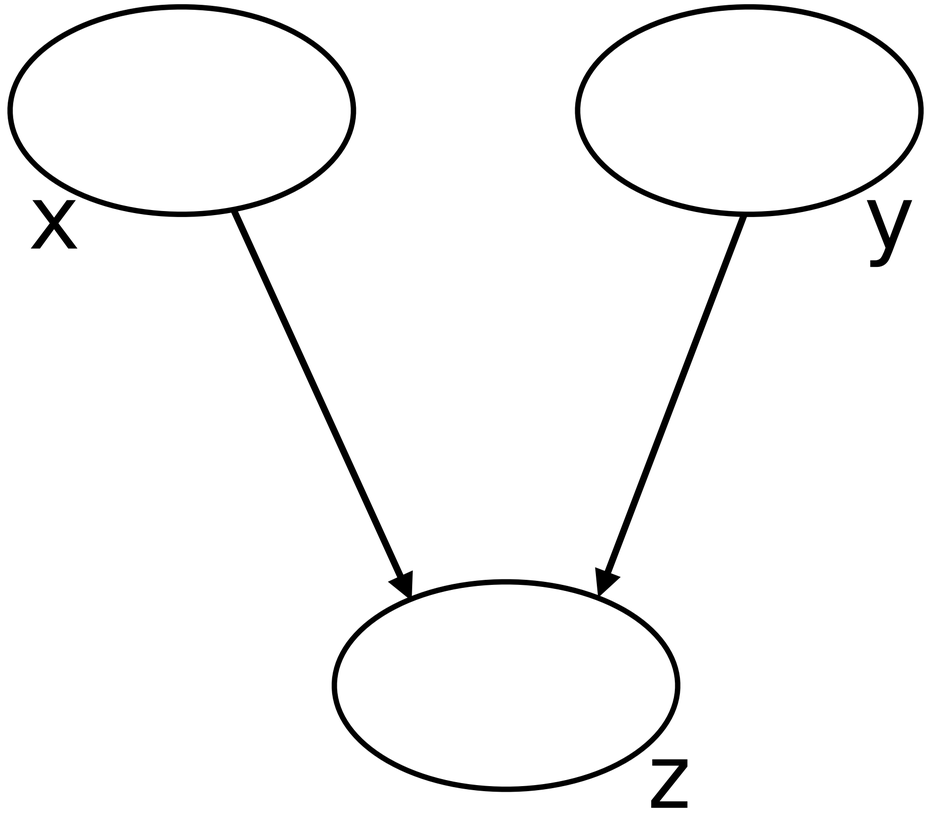
```
> C1 = 42
> C1 = 42
```

```
42
70
```

Dataflow computation

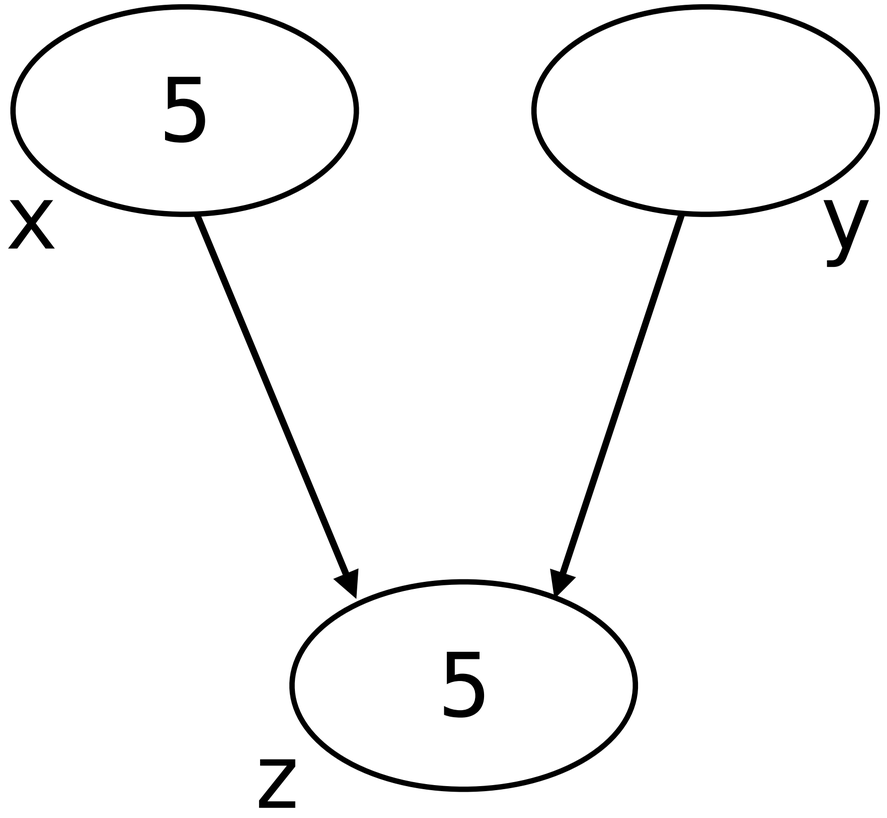


Operate **whenever one** updates

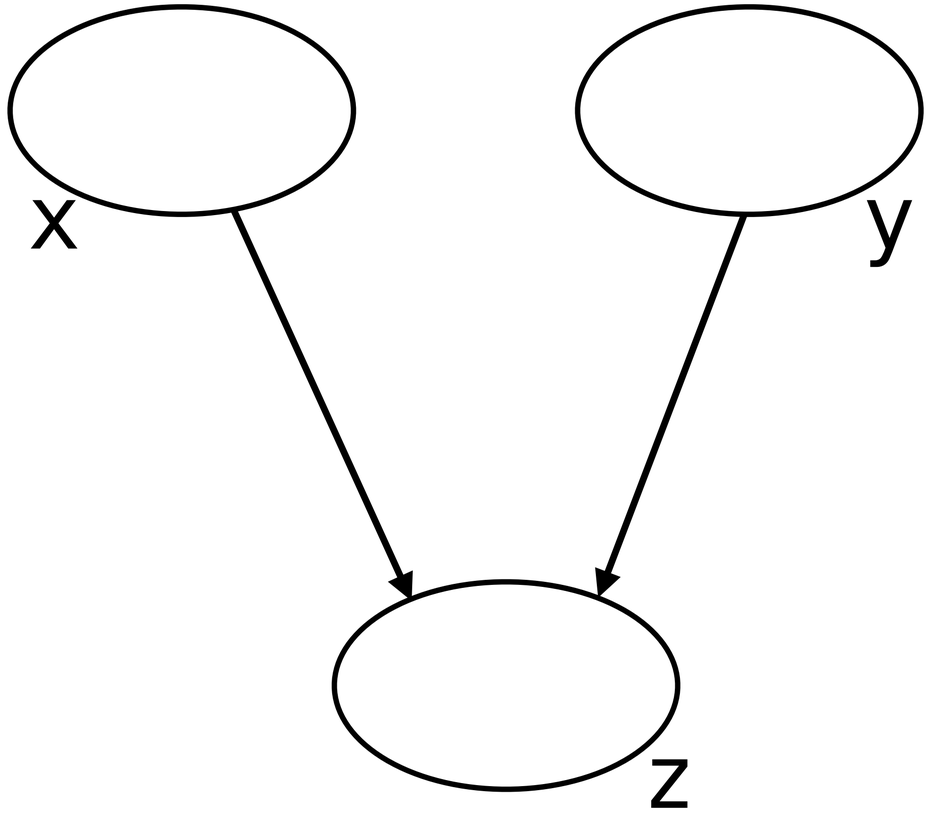


Operate **if all** update

Dataflow computation

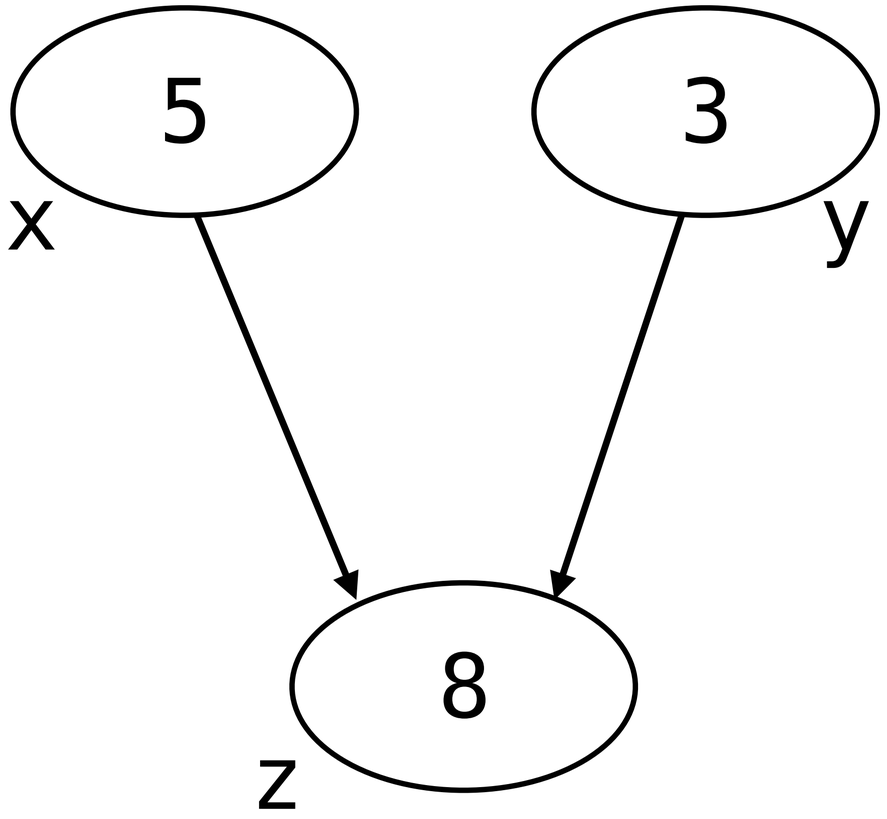


Operate **whenever one** updates

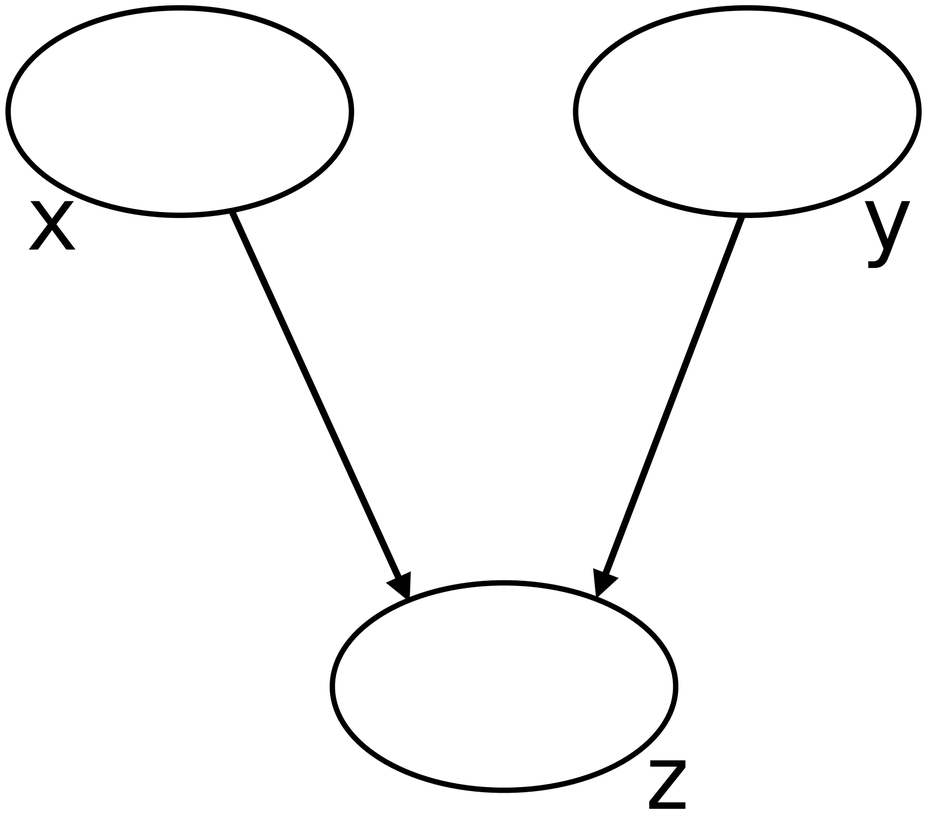


Operate **if all** update

Dataflow computation

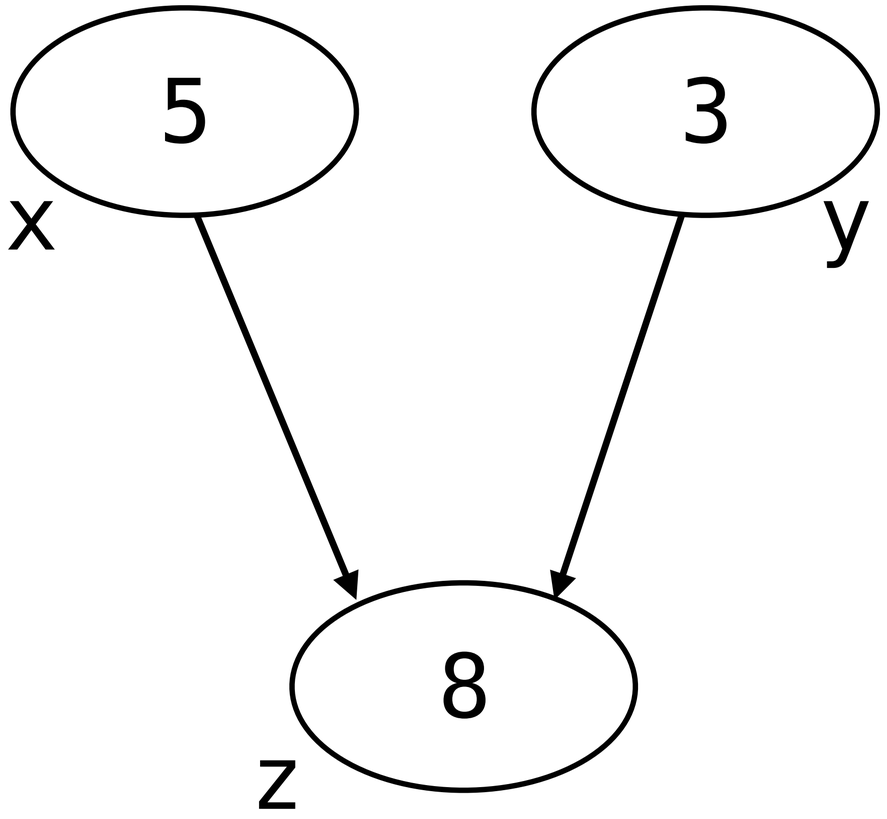


Operate **whenever one** updates

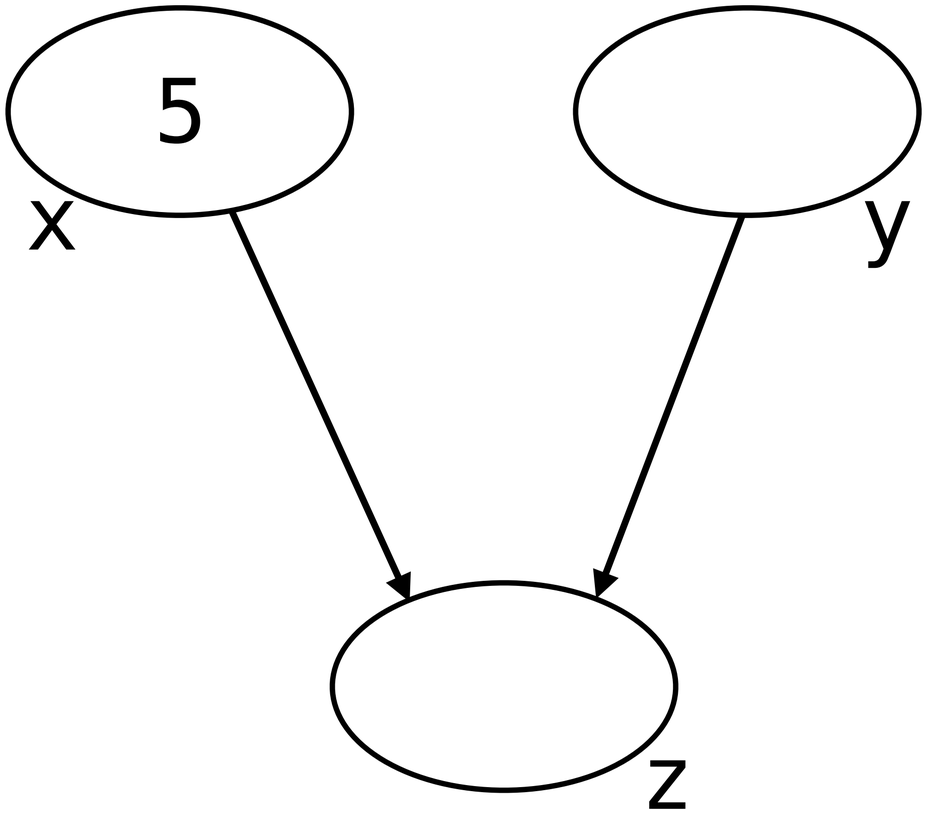


Operate **if all** update

Dataflow computation

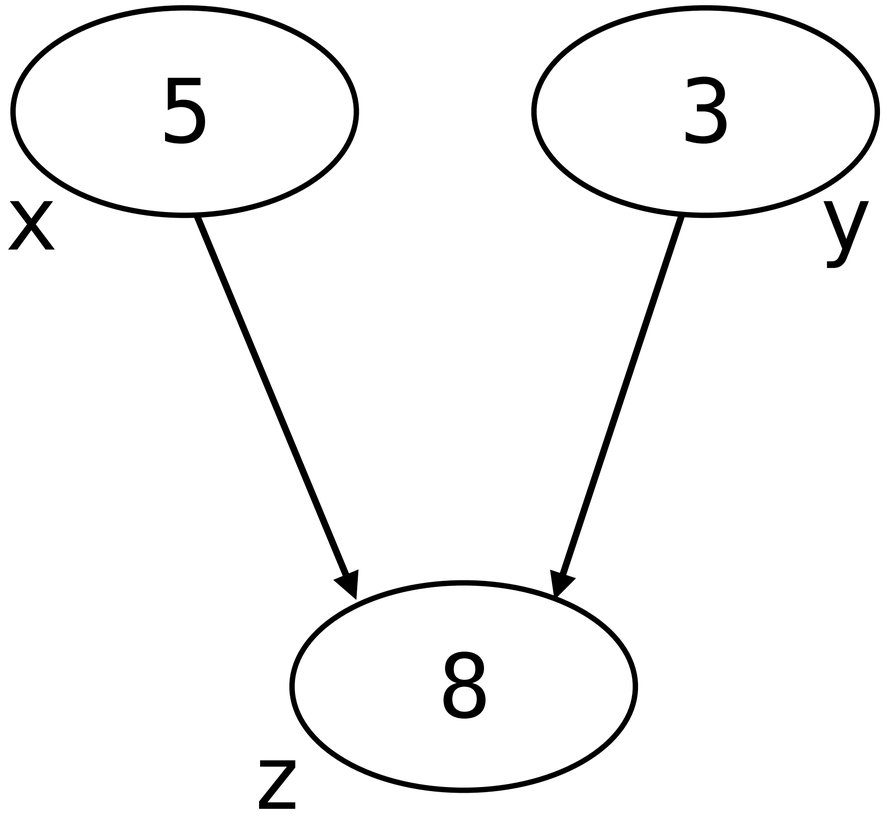


Operate **whenever one** updates

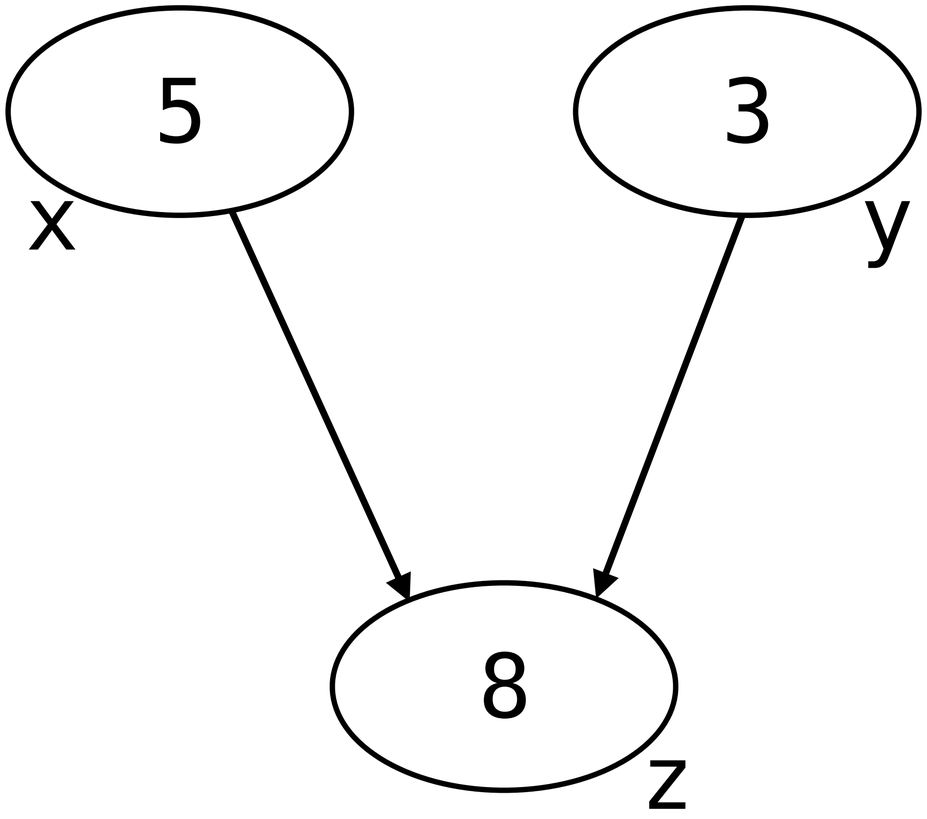


Operate **if all** update

Dataflow computation

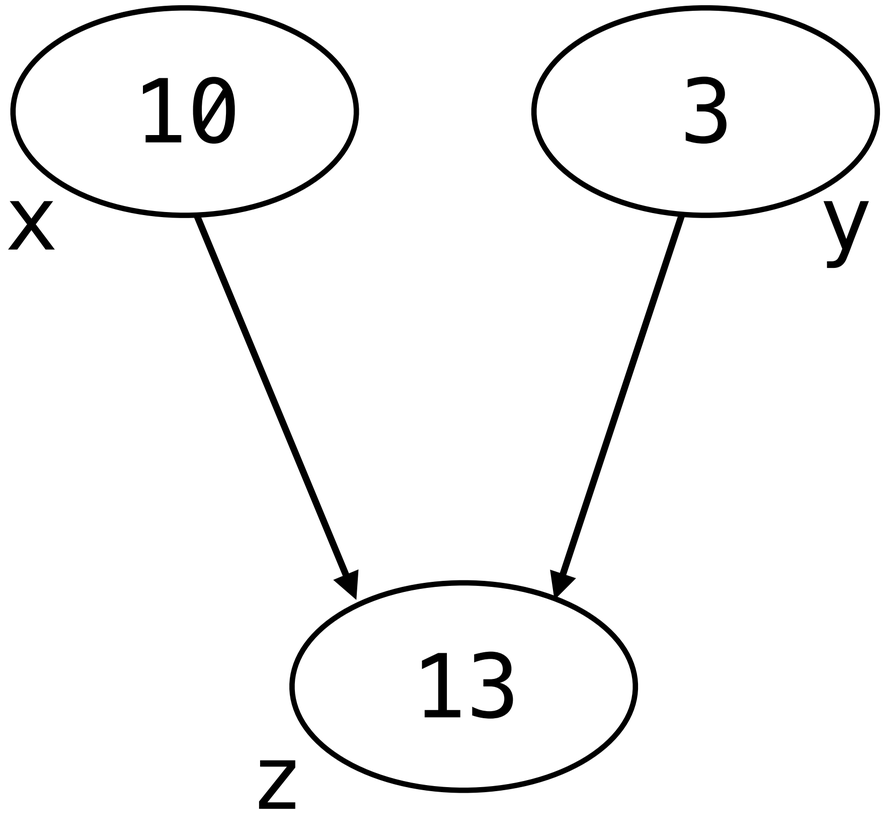


Operate **whenever one** updates

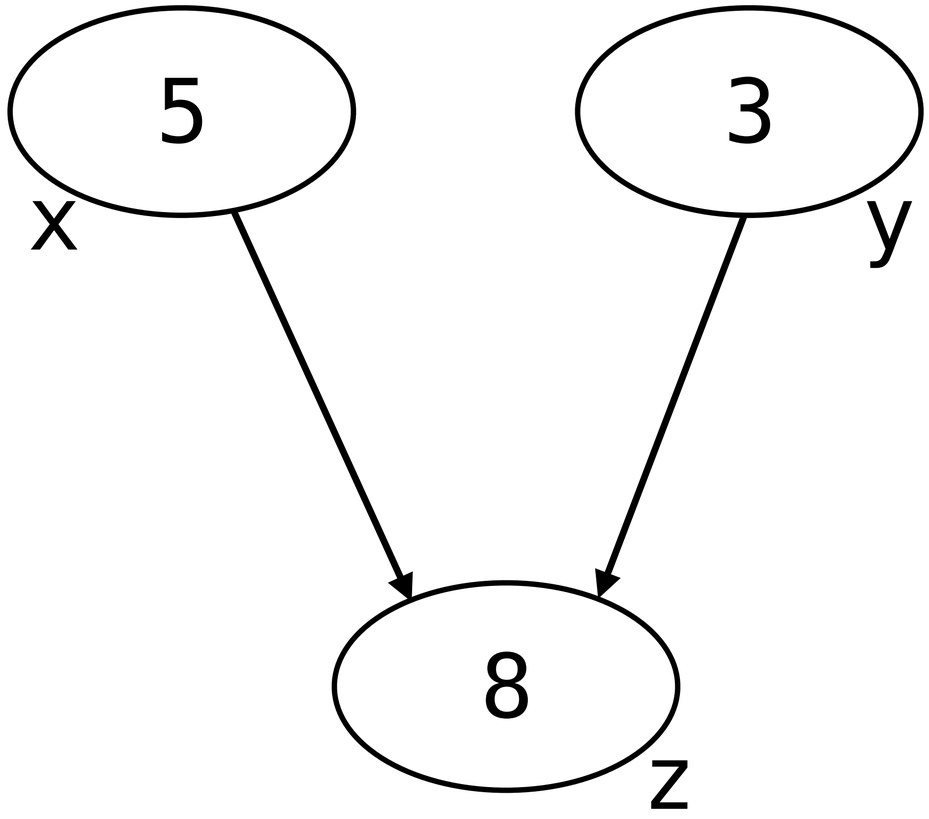


Operate **if all** update

Dataflow computation

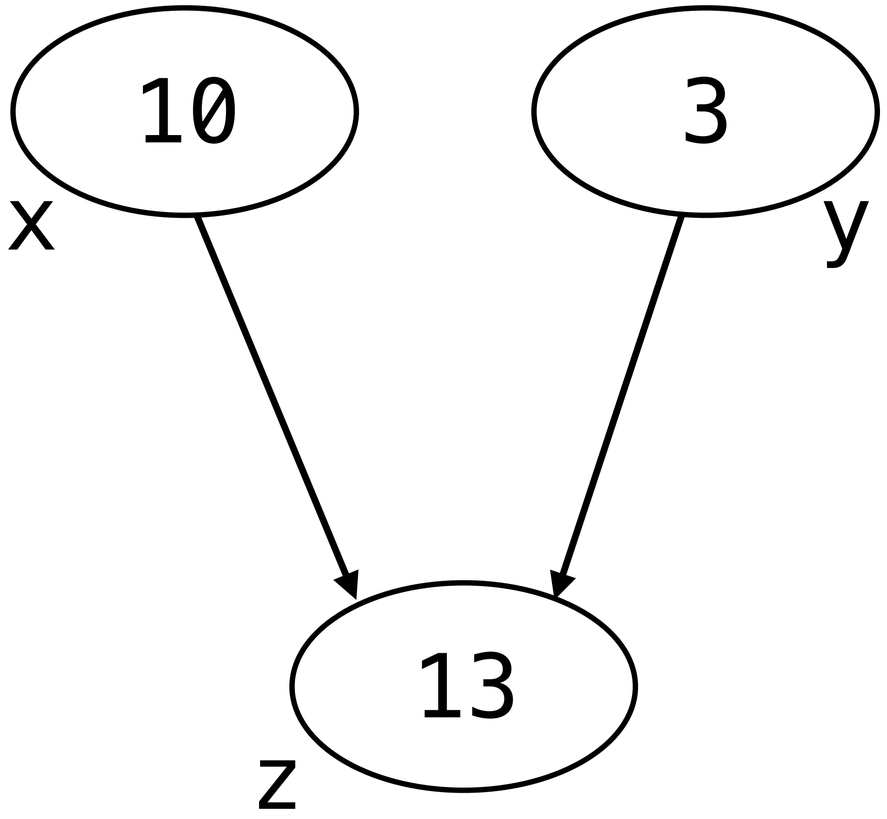


Operate **whenever one** updates

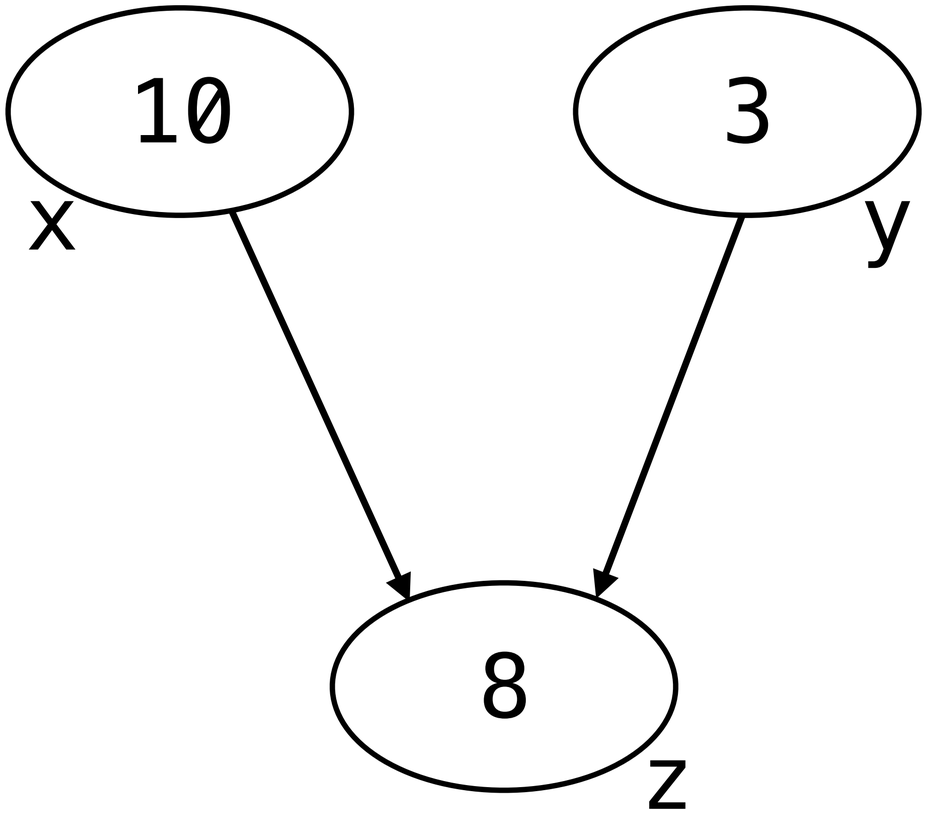


Operate **if all** update

Dataflow computation

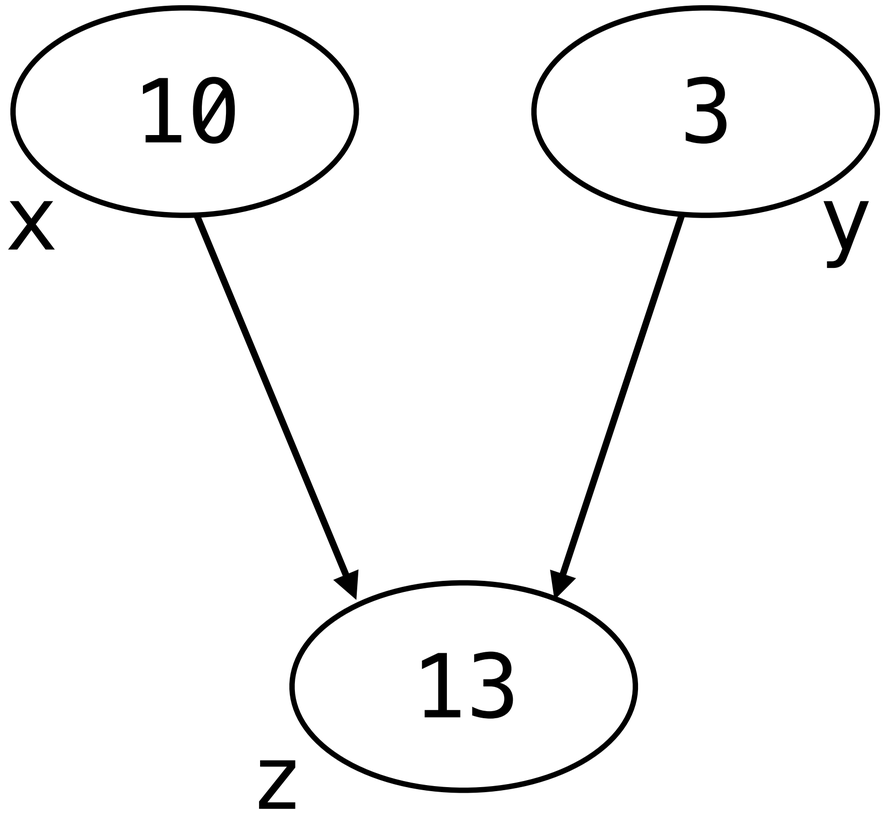


Operate **whenever one** updates

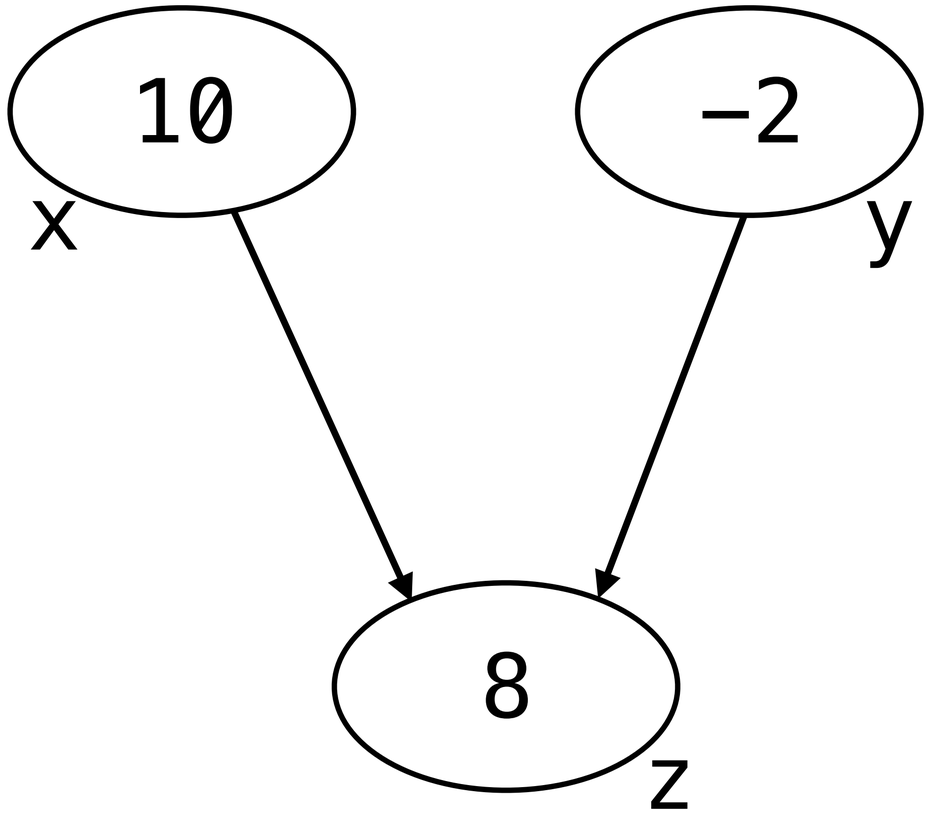


Operate **if all** update

Dataflow computation



Operate **whenever one** updates



Operate **if all** update

HOW TO CREATE AN INFINITE LIST



Streams

```
[1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 …]
```

```
fun {Generate N}
   N|{Generate N+1}
end
```

Streams

```
[1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 …]
```

```
fun {Generate N}
   N|{Generate N+1}
end
```



```
FATAL: The active memory
(732096606) after a GC is
over the maximal heap size
   threshold: 732096600
      Terminated VM 1
```

Streams

```
[1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 …]
```

```
fun {Generate N}
   N|{Generate N+1}
end
```



```
FATAL: The active memory
(732096606) after a GC is
over the maximal heap size
threshold: 732096600
Terminated VM 1
```

```
fun lazy {Generate2 N}
   N|{Generate2 N+1}
end
```

Streams

```
[1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 …]
```

```
fun {Generate N}
   N|{Generate N+1}
end
```



```
FATAL: The active memory
(732096606) after a GC is
over the maximal heap size
threshold: 732096600
Terminated VM 1
```

```
fun lazy {Generate2 N}
   N|{Generate2 N+1}
end
```



```
_<optimized>
```

Streams

```
[1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 …]
```

```
fun {Generate N}
   N|{Generate N+1}
end
```



```
FATAL: The active memory
(732096606) after a GC is
over the maximal heap size
threshold: 732096600
Terminated VM 1
```

```
fun lazy {Generate2 N}
   N|{Generate2 N+1}
end
```

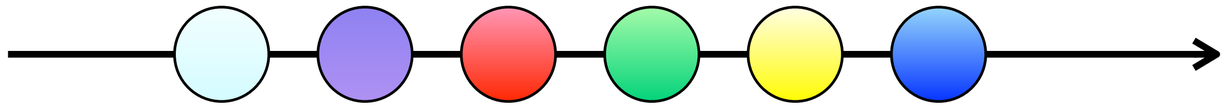


```
_<optimized>
```

won’t get a result **until used** it in an operation

Streams

- Un-ordered sequences of values
- No underlying data structure
- **Just a value**



```
scala> val s = Stream()
scala> s: scala.collection.immutable.Stream[Nothing] =
Stream()

scala> val s2 = Stream.empty
scala> s2: scala.collection.immutable.Stream[Nothing] =
Stream()

scala> val s3 = 1 #:: 2 #:: 3 #:: Stream.empty
scala: s3: scala.collection.immutable.Stream[Int] = Stream(1,
?)

scala> val s4 = (1 to 100000000).toStream
```

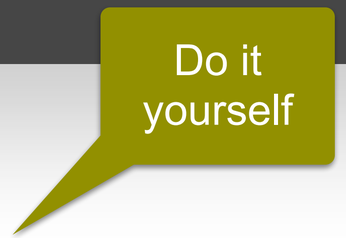




Streams

get odd numbers

```
fun lazy {Generate N}
   N|{Generate N+1}
end
```



Streams

get odd numbers

Do it yourself

```
fun lazy {Generate N}
   N|{Generate N+1}
end

fun {Odds X}
  local Ys in
   {List.take {Filter
      {Generate X}
      fun {$ X} X mod 2 ==1 end}
                3 Ys}
  end
end
```

Streams

get odd numbers

Do it yourself

```
fun lazy {Generate N}
   N|{Generate N+1}
end

fun {Odds X}
  local Ys in
   {List.take {Filter
      {Generate X}
      fun {$ X} X mod 2 ==1 end}
                3 Ys}
  end
end
```

Out of memory!
Why?

Streams

get odd numbers

Do it yourself

```
fun lazy {Generate N}
   N|{Generate N+1}
end


fun {Odds X}
   local Ys in
      Ys = {Generate X}
      {Show Ys.1|Ys.2.1|Ys.2.2.1}
   end
end
```

Streams

sum up a range

```
fun {Sum Xs A Limit}
   if Limit>0 then
      case Xs of X|Xr then
         {Sum Xr A+X Limit-1}
      end
   else A end
end
```

Streams

sum up a range

```
fun {Sum Xs A Limit}
   if Limit>0 then
      case Xs of X|Xr then
      {Sum Xr A+X Limit-1}
      end
   else A end
end
```

operating over the list

Streams

sum up a range

```
fun {Sum Xs A Limit}
   if Limit>0 then
      case Xs of X|Xr then
         {Sum Xr A+X Limit-1}
      end
   else A end
end

local Xs S in
   thread Xs={Generate 0} end
   S = {Sum Xs 0 15000000}
   {Show S}
end
```

operating over the list

On demand streams

primes’ generation

On demand streams

primes' generation

```
fun lazy {Sieve Xs}
   X|Xr = Xs in
    X|{Sieve {LFilter
         Xr fun {$ Y} Y mod X \= 0
     end}}
end
```

only keep the non multiples of other numbers

```
fun {Primes}
   {Sieve {Generate 2}}
end
```

Give me all the numbers that could be prime

On demand streams

- Transmission of information takes place through **buffers**, a **producer creates** data and puts it in a buffer for the **consumer to use**, both **processing** data at **different rates**
- If both the producer and the consumer are lazy, then all the **problems of transmission synchronization are solved**

On demand streams

```
fun {Buffer1 In N}
   thread
      End={List.drop In N}
   end
   fun lazy {Loop In End}
    thread E2 = End.2 end
     In.1|{Loop In.2 E2}
   end
in
   {Loop In End}
end
```

On demand streams

```
fun {Buffer1 In N}
   thread
      End={List.drop In N}
   end
   fun lazy {Loop In End}
    thread E2 = End.2 end
     In.1|{Loop In.2 E2}
   end
in
   {Loop In End}
end
```

produce up to the buffer size

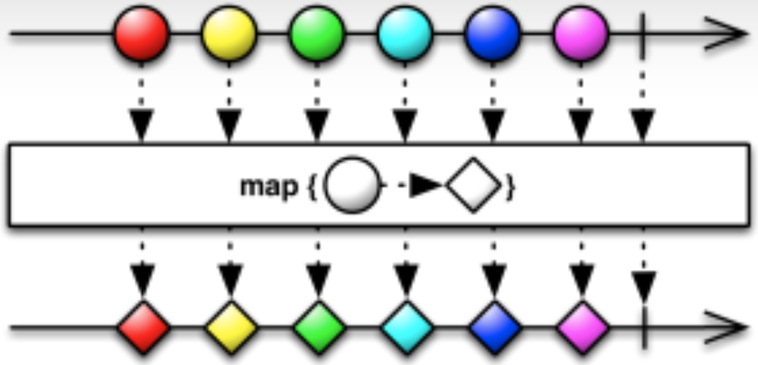
On demand streams

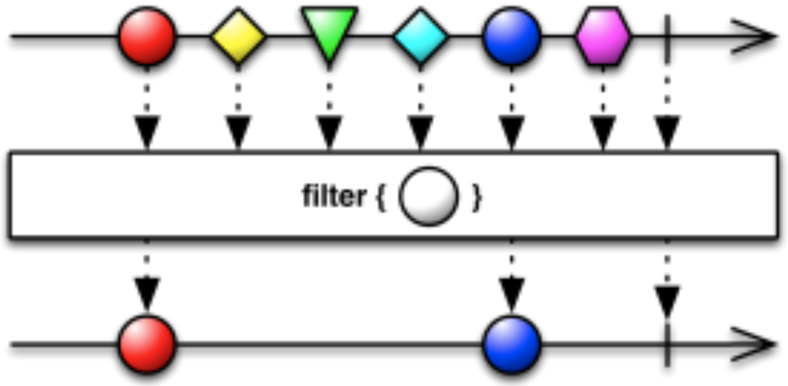
```
fun {Buffer1 In N}
   thread
      End={List.drop In N}
   end
   fun lazy {Loop In End}
    thread E2 = End.2 end
     In.1|{Loop In.2 E2}
   end
in
   {Loop In End}
end
```

produce up to the buffer size

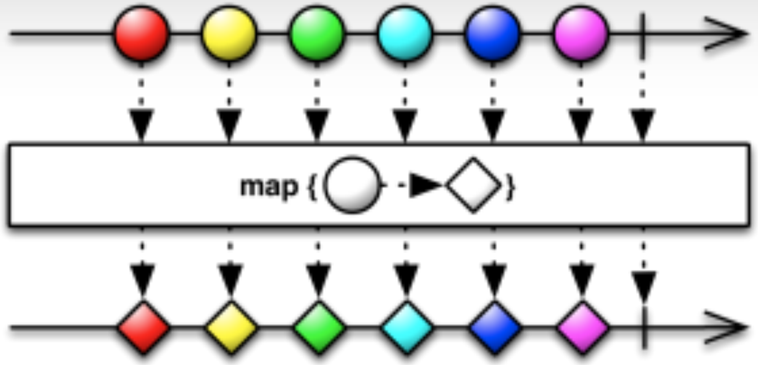
Consume and fill

On demand streams

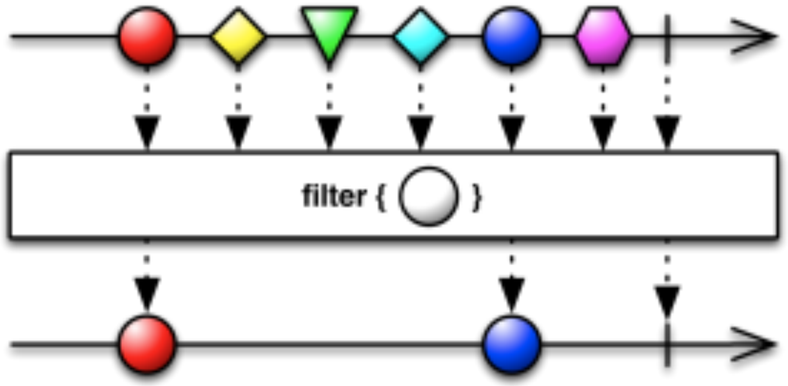




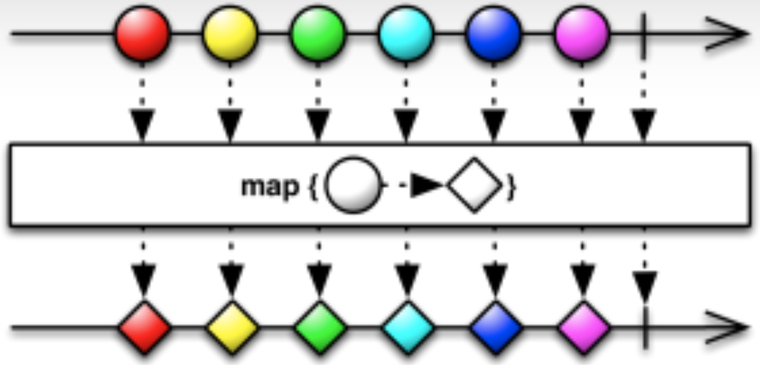
On demand streams



No lifting of primitive functions

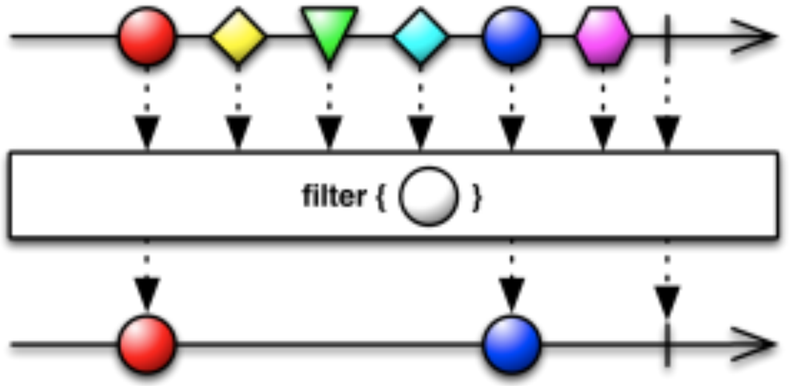


On demand streams



No lifting of primitive functions

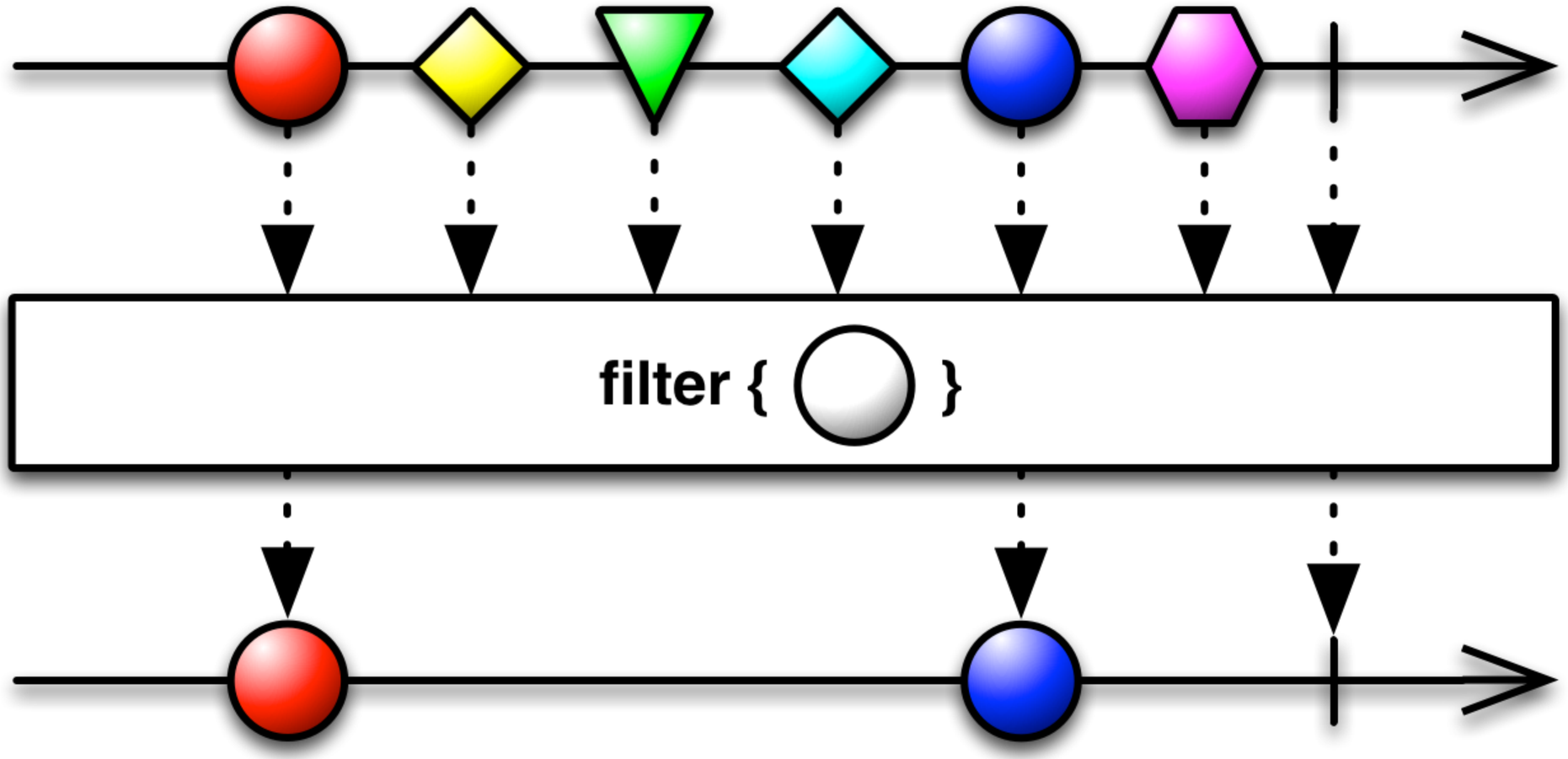
List processing operations



IF WE OPERATE OVER LAZY LISTS, WE REQUIRE LAZY OPERATIONS TO MANGE THEM



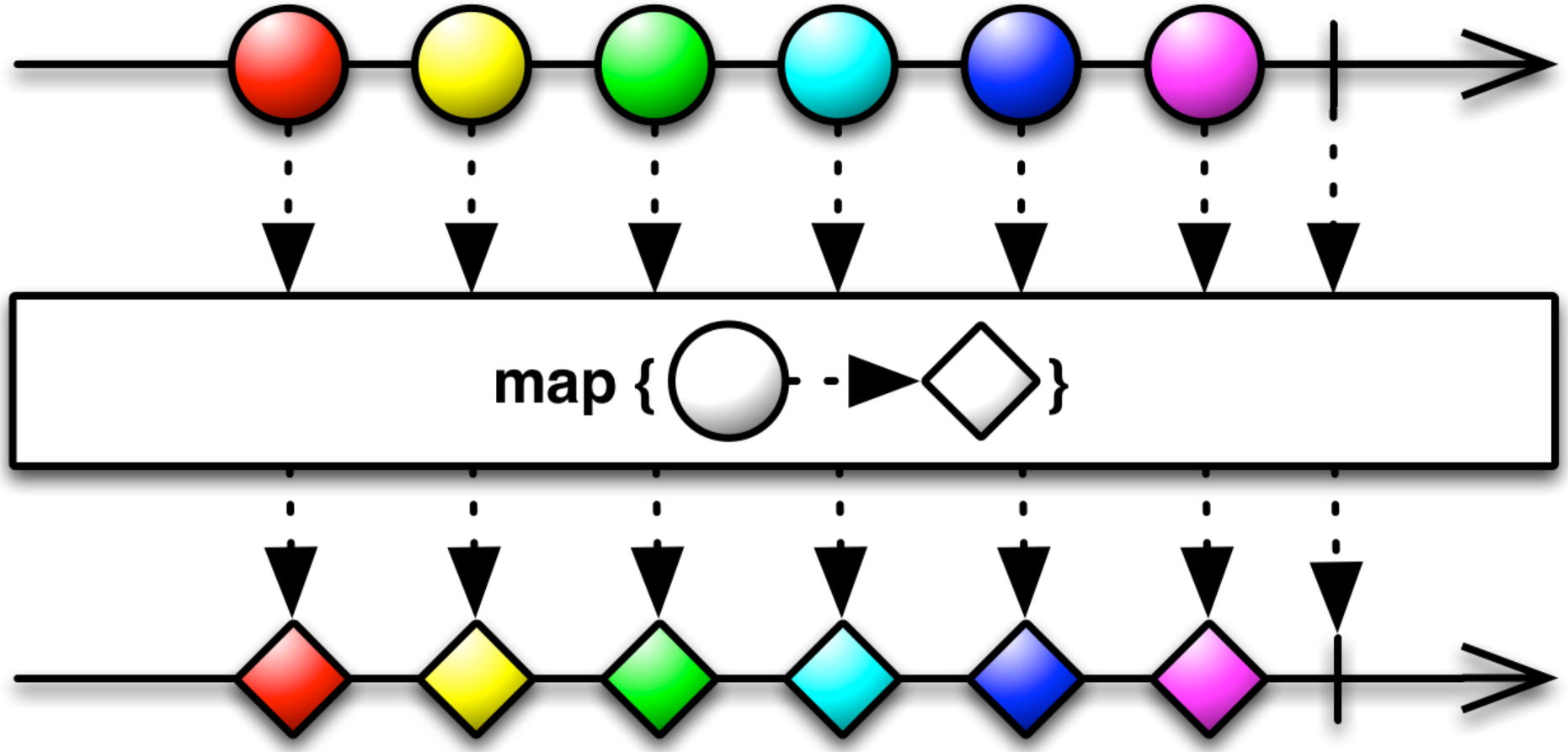
On demand streams



Lazy filter

```
fun lazy {LFilter Xs F}
   case Xs of nil then nil
   [] X|Xr then
   if {F X} then X|{LFilter Xr F}
   else {LFilter Xr F} end
   end
end
```

On demand streams



Lazy map

```
fun lazy {LMap Xs F}
   case Xs of nil then nil
   [] X|Xr then {F X}|{LMap Xr F}
   end
end
```

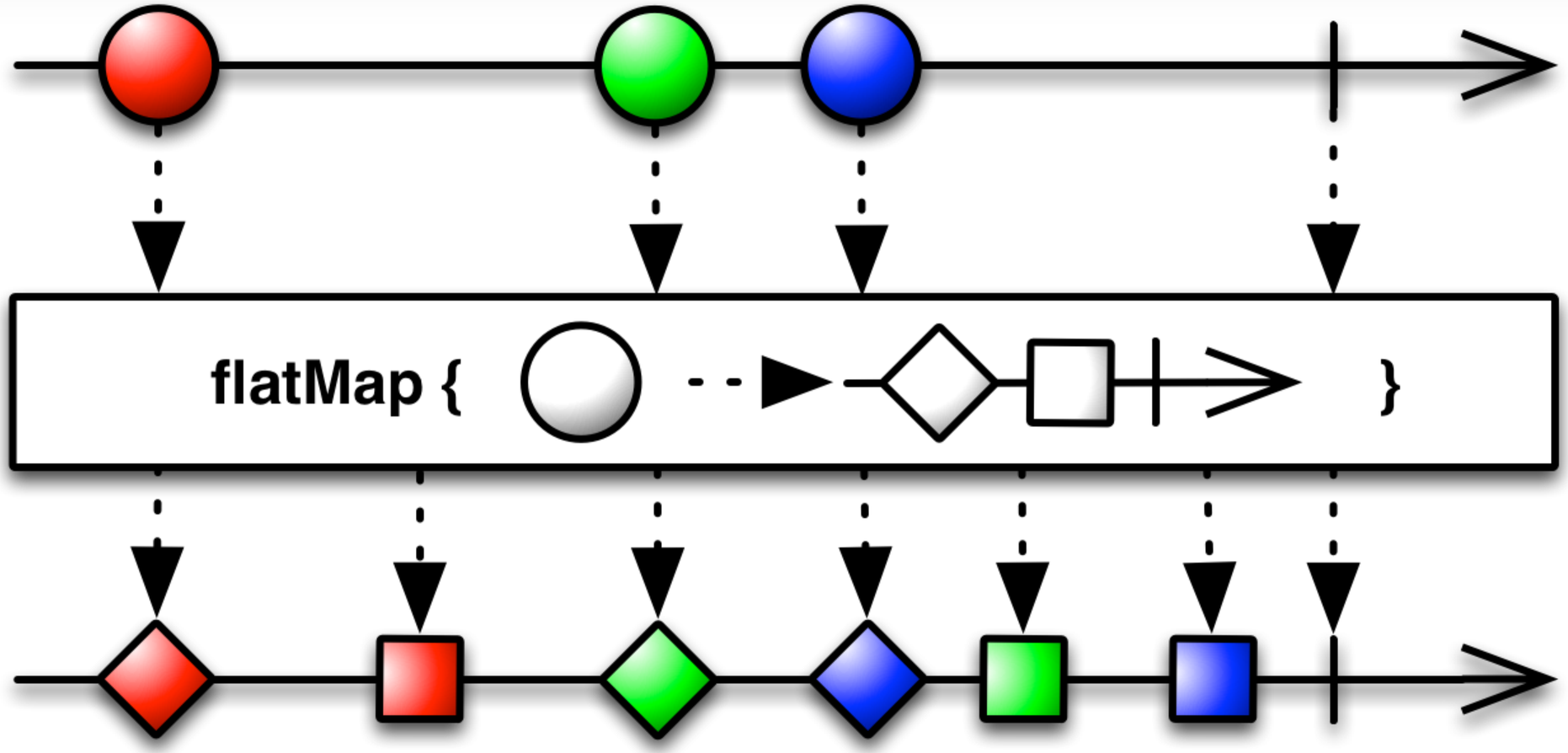
Streams

```
[1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 …]
```

```
Xs = 1|{LMap Xs fun {$ X} X+1 end}
```



On demand streams



Lazy flatmap

```
fun lazy {LFlatmap Xs F}
   case Xs of nil then nil
   [] X|Xr then
    {Append {F X} {LFaltmap Xr F}}
    end
end
```

All even Fibonacci numbers

Putting lazy higher order functions to work

Do it yourself

All even Fibonacci numbers

Putting lazy higher order functions to work

Do it yourself

```
fun {EvenFib N}
 {LFoldR
  {LFilter
   {LMap {fun lazy {Generate X}
      X|{Generate X+1} end 0}
    fun {Fib X}
     if X <= 1 then 1
     else {Fib X-1} {Fib X-2} end
    end}
   fun {$ X} X mod ==0 end}
 fun {$ X Y} X|Y end nil}
end
```